

Contenido

Module_btree	1
Module_ordinal_user	21
linkedListAlbum.....	33
linkedListImages.....	40
Module_admin_user	45
Module_layer	50
Module_abbtree_layers.....	54
Module_avlTree_images.....	62
Module_Queue	80
Module_Matrix.....	88

Module_btree

Primero, se define un `queue_node`, que es un nodo de una cola que se utilizará para recorrer el árbol B. Este nodo tiene un puntero a un árbol B (`tree`) y un puntero al siguiente nodo en la cola (`next`).

```
1  type queue_node
2      type(BTree), pointer :: tree => null()
3      type(queue_node), pointer :: next => null()
4  end type queue_node
```

Luego, se define una `queue_BTree`, que es una cola de nodos de árbol B. Esta cola tiene un puntero al primer nodo (`head`) y un puntero al último nodo (`tail`). También contiene dos procedimientos: `queueaddBTree` para agregar un árbol B a la cola y `dequeueBTree` para eliminar un árbol B de la cola.



```
1  type queue_BTree
2      type(queue_node), pointer :: head => null()
3      type(queue_node), pointer :: tail => null()
4  contains
5      procedure :: queueaddBTree
6      procedure :: dequeueBTree
7  end type
```

Después, se define un nodeptr, que es un puntero a un nodo de un árbol B.



```
1  type nodeptr
2      type (BTree), pointer :: ptr => null()
3  end type nodeptr
```

Finalmente, se define el tipo BTree, que representa un árbol B. Este tipo tiene varias propiedades y procedimientos. Las propiedades incluyen MAXI y MINI, que representan el número máximo y mínimo de hijos que puede tener un nodo del árbol, respectivamente. También tiene una matriz val de tipo ordinal_user, que almacena los valores en los nodos del árbol, y una matriz link de tipo nodeptr, que almacena punteros a los hijos de los nodos. Además, tiene un puntero root al nodo raíz del árbol.

Los procedimientos en BTree incluyen operaciones comunes en árboles B como insert (insertar), remove (eliminar), y traversal (recorrido). También incluye otros procedimientos específicos como getUser y getUserAdmin, que probablemente se utilizan para obtener información de usuario en el contexto de este código.



```
1  type BTree
2      !orden=MAXI+1
3      integer :: MAXI = 4
4      !el numero debe de ser el mismo que el maxi
5      integer :: MINI = ceiling((dble(4)-1)/2)
6      ! val(0:n), n=maxi+1
7      type(ordinal_user) :: val(0:5)
8      integer :: num = 0
9      ! link(0:n), n=maxi+1
10     type(nodeptr) :: link(0:5)
11     type(BTree), pointer :: root => null()
12     contains
13     procedure :: insert
14     procedure :: returnRoot
15     procedure :: traversal
16     procedure :: traversal2
17     procedure :: remove
18     procedure :: graphBTree
19     procedure :: setValue
20     procedure :: splitNode
21     procedure :: createNode
22     procedure :: deletenode
23     procedure :: getUser
24     procedure :: getUserAdmin
25     procedure :: breadthFirstMatrix_adminReport
26 end type BTree
27
```

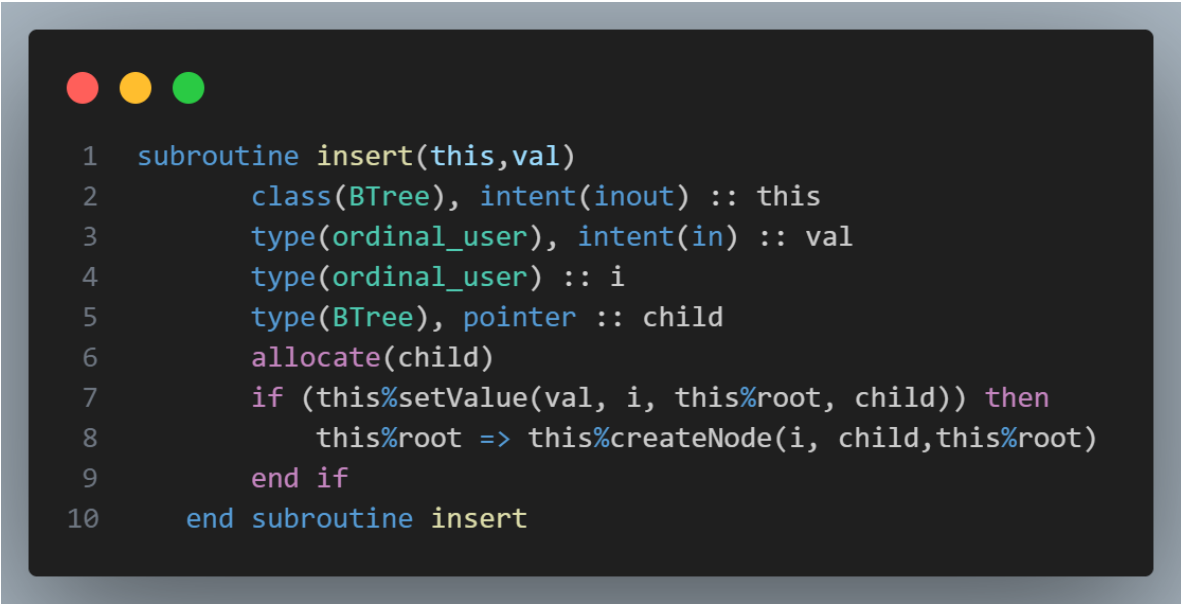
La subrutina insert toma dos argumentos: this y val. this es una instancia de la clase BTree que se pasa por referencia (intent(inout)), lo que significa que cualquier cambio que se haga a this dentro de la subrutina se reflejará fuera de ella. val es un valor de

tipo `ordinal_user` que se pasa por valor (`intent(in)`), lo que significa que cualquier cambio que se haga a `val` dentro de la subrutina no se reflejará fuera de ella.

Dentro de la subrutina, se declara una variable `i` de tipo `ordinal_user` y un puntero `child` a un `BTree`. Luego, se asigna memoria para `child` con la instrucción `allocate`.

Después, se llama al método `setValue` de `this` con `val`, `i`, `this%root`, y `child` como argumentos. Si `setValue` devuelve `true`, entonces se llama al método `createNode` de `this` con `i`, `child`, y `this%root` como argumentos, y el resultado se asigna a `this%root`. Esto significa que se crea un nuevo nodo en el árbol con `i` como valor y `child` como hijo, y este nuevo nodo se convierte en la nueva raíz del árbol.

Finalmente, la subrutina `insert` termina, habiendo insertado un nuevo valor en el árbol B.

A screenshot of a code editor with a dark background and light-colored text. The code is a Fortran subroutine named 'insert'. It takes 'this' and 'val' as arguments. Inside, it declares 'i' as an ordinal_user and 'child' as a pointer to a BTree. It then allocates memory for 'child', calls 'setValue' on 'this' with 'val', 'i', 'this%root', and 'child' as arguments. If 'setValue' returns true, it calls 'createNode' on 'this' with 'i', 'child', and 'this%root' as arguments, and assigns the result to 'this%root'. Finally, it ends the subroutine.

```
1  subroutine insert(this,val)
2      class(BTree), intent(inout) :: this
3      type(ordinal_user), intent(in) :: val
4      type(ordinal_user) :: i
5      type(BTree), pointer :: child
6      allocate(child)
7      if (this%setValue(val, i, this%root, child)) then
8          this%root => this%createNode(i, child, this%root)
9      end if
10     end subroutine insert
```

La función `returnRoot` toma un argumento `this` de tipo `BTree` y devuelve un puntero `myRoot` al nodo raíz del árbol B. Esta función es bastante simple: simplemente asigna el nodo raíz de `this` a `myRoot`.



```
1  function returnRoot(this) result(myRoot)
2      class(BTree) :: this
3      type(BTree), pointer :: myRoot
4      myRoot => this%root
5  end function returnRoot
```

La función `setValue` es más compleja. Es una función recursiva que intenta establecer un valor en un nodo específico del árbol B. Toma cinco argumentos: `this` (una instancia de `BTree`), `val` (el valor a establecer), `pval` (un valor de salida), `node` (el nodo en el que se intenta establecer el valor), y `child` (un nodo hijo). La función devuelve un valor lógico `res` que indica si la operación fue exitosa.

Dentro de `setValue`, se declara una variable `pos` que se utilizará para rastrear la posición en el nodo, y un puntero `newnode` a un `BTree`. Luego, se asigna memoria para `newnode`.

La función primero verifica si `node` está asociado (es decir, si apunta a algo). Si no lo está, entonces `pval` se establece en `val`, `child` se establece en `null`, `res` se establece en `.true.`, y la función retorna.

Luego, la función verifica si `val%DPI` es menor que `node%val(1)%DPI`. Si es así, `pos` se establece en 0. Si no, `pos` se establece en `node%num` y se entra en un bucle que decrementa `pos` hasta que `val%DPI` no sea menor que `node%val(pos)%DPI` o `pos` sea 1. Si `val%DPI` es igual a `node%val(pos)%DPI`, entonces se imprime un mensaje de error, `res` se establece en `.false.`, y la función retorna.

Finalmente, la función llama a sí misma recursivamente con `val`, `pval`, `node%link(pos)%ptr`, y `child` como argumentos. Si esta llamada devuelve `.true.`, entonces se verifica si `node%num` es menor que `this%MAXI`. Si es así, se llama a la subrutina `insertNode`. Si no, se llama a la subrutina `splitNode`, `child` se establece en `newnode`, `res` se establece en `.true.`, y la función retorna. Si la llamada recursiva a `setValue` devuelve `.false.`, entonces `res` se establece en `.false.` y la función retorna.

```

1 recursive function setValue(this, val, pval, node, child) result(res)
2     class(BTree), intent(inout) :: this
3     type(ordinal_user), intent(in) :: val
4     type(ordinal_user), intent(inout) :: pval
5     type(BTree), pointer, intent(inout) :: node
6     type(BTree), pointer, intent(inout) :: child
7     type(BTree), pointer :: newnode
8     integer :: pos
9     logical :: res
10    allocate(newnode)
11    if (.not. associated(node)) then
12        pval = val
13        child => null()
14        res = .true.
15        return
16    end if
17    if (val%DPI < node%val(1)%DPI) then
18        pos = 0
19    else
20        pos = node%num
21        do while (val%DPI < node%val(pos)%DPI .and. pos > 1)
22            pos = pos - 1
23        end do
24        if (val%DPI == node%val(pos)%DPI) then
25            print *, "Duplicates are not permitted"
26            res = .false.
27            return
28        end if
29    end if
30    if (this%setValue(val, pval, node%link(pos)%ptr, child)) then
31        if (node%num < this%MAXI) then
32            call insertNode(pval, pos, node, child)
33        else
34            call this%splitNode(pval, pval, pos, node, child, newnode)
35            child => newnode
36            res = .true.
37            return
38        end if
39    end if
40    res = .false.
41 end function setValue

```

La subrutina insertNode toma cuatro argumentos: val, pos, node y child. val es un valor de tipo ordinal_user que se va a insertar. pos es un entero que indica la posición en la que se va a insertar val. node es un puntero a un BTree que se pasa por referencia (intent(inout)), lo que

significa que cualquier cambio que se haga a node dentro de la subrutina se reflejará fuera de ella. child es un puntero a un BTree que representa el hijo del nodo que se va a insertar.

Dentro de la subrutina, se declara una variable j que se inicializa con el número de elementos en node (node%num). Luego, se entra en un bucle que se ejecuta mientras j sea mayor que pos. Dentro del bucle, se desplazan los valores y los enlaces en node una posición hacia la derecha (desde j hasta pos), y se decrementa j.

Después del bucle, se inserta val en la posición j + 1 de node y se establece el enlace en la posición j + 1 de node a child. Finalmente, se incrementa el número de elementos en node.

En resumen, la subrutina insertNode inserta un valor y un hijo en una posición específica de un nodo en un árbol B, desplazando los valores y los hijos existentes a la derecha si es necesario.

```
1  subroutine insertNode(val, pos, node, child)
2      type(ordinal_user), intent(in) :: val
3      integer, intent(in) :: pos
4      type(BTree), pointer, intent(inout) :: node
5      type(BTree), pointer, intent(in) :: child
6      integer :: j
7      j = node%num
8      do while (j > pos)
9          node%val(j + 1) = node%val(j)
10         node%link(j + 1)%ptr => node%link(j)%ptr
11         j = j - 1
12     end do
13     node%val(j + 1) = val
14     node%link(j + 1)%ptr => child
15     node%num = node%num + 1
16 end subroutine insertNode
```

La subrutina splitNode se utiliza para dividir un nodo en un árbol B (BTree) cuando está lleno. Toma siete argumentos: this (una instancia de BTree), val (un valor a insertar), pval (un valor de salida), pos (la posición en la que se intenta insertar val), node (el nodo que se va a dividir), child (un nodo hijo), y newnode (un nuevo nodo que se creará si es necesario).

Dentro de splitNode, se declaran tres variables enteras: median, i, y j. median se establece en this%MINI + 1 si pos es mayor que this%MINI, y en this%MINI en caso contrario. Luego,

si newnode no está asociado (es decir, no apunta a nada), se le asigna memoria y se establecen todos sus enlaces a null.

Después, se copian los valores y los enlaces de node a newnode, comenzando desde median + 1 hasta this%MAXI. Luego, se establece el número de elementos en node a median y en newnode a this%MAXI - median.

A continuación, si pos es menor o igual que this%MINI, se llama a la subrutina insertNode con val, pos, node, y child como argumentos. Si pos es mayor que this%MINI, se llama a insertNode con val, pos - median, newnode, y child como argumentos.

Finalmente, pval se establece en el último valor de node, el primer enlace de newnode se establece en el último enlace de node, y se decrementa el número de elementos en node.



```
1  subroutine splitNode(this, val, pval, pos, node, child, newnode)
2      class(BTree), intent(in) :: this
3      type(ordinal_user), intent(in) :: val
4      integer, intent(in) :: pos
5      type(ordinal_user), intent(inout) :: pval
6      type(BTree), pointer, intent(inout) :: node, newnode
7      type(BTree), pointer, intent(in) :: child
8      integer :: median, i, j
9      if (pos > this%MINI) then
10         median = this%MINI + 1
11     else
12         median = this%MINI
13     end if
14     if (.not. associated(newnode)) then
15         allocate(newnode)
16         do i = 0, this%MAXI
17             newnode%link(i)%ptr => null()
18         enddo
19     end if
20     j = median + 1
21     do while (j <= this%MAXI)
22         newnode%val(j - median) = node%val(j)
23         newnode%link(j - median)%ptr => node%link(j)%ptr
24         j = j + 1
25     end do
26     node%num = median
27     newnode%num = this%MAXI - median
28     if (pos <= this%MINI) then
29         call insertNode(val, pos, node, child)
30     else
31         call insertNode(val, pos - median, newnode, child)
32     end if
33     pval = node%val(node%num)
34     newnode%link(0)%ptr => node%link(node%num)%ptr
35     node%num = node%num - 1
36 end subroutine splitNode
```

La función createNode se utiliza para crear un nuevo nodo en un árbol B. Toma cuatro argumentos: this (una instancia de BTree), val (un valor a insertar), child (un nodo hijo), y root (el nodo raíz del árbol B). La función devuelve un puntero newNode a un BTree.

Dentro de createNode, se asigna memoria para newNode, se establece su primer valor en val, su número de elementos en 1, su primer enlace en root, y su segundo enlace en child. Luego, se establecen todos los enlaces restantes en newNode a null.

```
1  function createNode(this, val, child, root) result(newNode)
2      class(BTree), intent(in) :: this
3      type(ordinal_user), intent(in) :: val
4      type(BTree), pointer, intent(in) :: child
5      type(BTree), pointer, intent(in) :: root
6      type(BTree), pointer :: newNode
7      integer :: i
8      allocate(newNode)
9      newNode%val(1) = val
10     newNode%num = 1
11     newNode%link(0)%ptr => root
12     newNode%link(1)%ptr => child
13     do i = 2, this%MAXI
14         newNode%link(i)%ptr => null()
15     end do
16 end function createNode
```

La subrutina traversal es una función recursiva que recorre un árbol B (BTree) y escribe los valores de los nodos en la salida estándar. Toma dos argumentos: this (una instancia de BTree) y myNode (el nodo actual en el recorrido). Si myNode está asociado (es decir, no es null), se escribe un corchete de apertura en la salida estándar, luego se recorren los valores en myNode y se escriben en la salida estándar. Después, se recorren los enlaces en myNode y se llama a traversal en cada uno de ellos. Finalmente, se escribe un corchete de cierre en la salida estándar.

```

1  recursive subroutine traversal(this,myNode)
2      class(BTree), intent(in) :: this
3      type(BTree), pointer, intent(in) :: myNode
4      integer :: i
5      if (associated(myNode)) then
6          write (*, '(A)', advance='no') ' ['
7          !en el i=0 no hay nada
8          do i = 1, myNode%num
9              write (*, '(I17)', advance='no') myNode%val(i)%DPI
10             end do
11             do i = 0, myNode%num
12                 ! write (*, '(I5)', advance='no') i
13                 call this%traversal(myNode%link(i)%ptr)
14             end do
15             write (*, '(A)', advance='no') ' ] '
16         end if
17     end subroutine traversal

```

La subrutina traversal2 es similar a traversal, pero además de escribir el valor DPI de cada nodo, también escribe su password entre paréntesis.

```

1 recursive subroutine traversal2(this,myNode)
2     class(BTree), intent(in) :: this
3     type(BTree), pointer, intent(in) :: myNode
4     integer :: i
5     if (associated(myNode)) then
6         write (*, '(A)', advance='no') ' ['
7         !en el i=0 no hay nada
8         do i = 1, myNode%num
9             write (*, '(I17)', advance='no') myNode%val(i)%DPI
10            write (*, '(A)', advance='no') "("
11            write (*, '(A)', advance='no') myNode%val(i)%password
12            write (*, '(A)', advance='no') ")"
13        end do
14        do i = 0, myNode%num
15            ! write (*, '(I5)', advance='no') i
16            call this%traversal(myNode%link(i)%ptr)
17        end do
18        write (*, '(A)', advance='no') ' ] '
19    end if
20 end subroutine traversal2

```

La subrutina remove se utiliza para eliminar un nodo de un árbol B. Toma dos argumentos: this (una instancia de BTree) y DPI (el valor del nodo a eliminar). Primero, se asigna memoria para un nuevo nodo root2. Luego, se llama a la subrutina deletenode en this%root con DPI como argumento, y el resultado se almacena en root2. Después, se llama a traversal en root2%returnRoot() para imprimir el árbol después de la eliminación. Finalmente, se libera la memoria de this%root y se establece this%root en root2%returnRoot().



```

1  subroutine remove(this,DPI)
2      class(BTree), intent(inout) :: this
3      integer(kind=8), intent(in) :: DPI
4      class(BTree), pointer :: root2
5      allocate(root2)
6      print *, ""
7      call this%deletenode(DPI,this%root,root2)
8      call root2%traversal(myNode=root2%returnRoot())
9      print *, ""
10     deallocate(this%root)
11     this%root => root2%returnRoot()
12 end subroutine remove

```

La subrutina deletenode es una función recursiva que se utiliza para eliminar un nodo de un árbol B (BTree). Toma cuatro argumentos: this (una instancia de BTree), DPI (el valor del nodo a eliminar), root1 (el nodo raíz del árbol) y tree (un árbol B donde se insertarán los nodos que no se eliminan). Si root1 está asociado (es decir, no es null), se recorren los valores en root1. Si el valor DPI de un nodo no es igual al DPI proporcionado, se inserta el nodo en tree. Luego, se llama a deletenode en cada enlace en root1.

```

1  recursive subroutine deletenode(this,DPI,root1,tree)
2      class(BTree), intent(inout) :: this
3      integer(kind=8), intent(in) :: DPI
4
5      type(BTree), pointer ,intent(out) :: root1
6      class(BTree), pointer, intent(out) :: tree
7      integer :: i
8      if (associated(root1)) then
9          do i = 1, root1%num
10             if (DPI /= root1%val(i)%DPI) then
11                 call tree%insert(root1%val(i))
12             else
13                 end if
14             end do
15             i = 0
16             do i = 0, root1%num
17                 call this%deletenode(DPI,root1%link(i)%ptr,tree)
18             end do
19         end if
20     end subroutine deletenode

```

La subrutina graphBTree se utiliza para generar un gráfico de un árbol B. Toma dos argumentos: this (una instancia de BTree) y myNode (el nodo raíz del árbol). Primero, se abre un archivo llamado "treeUsers.dot" para escritura. Si myNode no está asociado, se escribe en el archivo que el árbol está vacío. Si myNode está asociado, se recorren los valores en myNode y se escriben en el archivo en formato de gráfico. Luego, se recorren los enlaces en myNode y se llama a graphrec en cada uno de ellos. Finalmente, se cierra el archivo y se llama a execute_command_line para generar una imagen del gráfico utilizando Graphviz.

```

1 recursive subroutine graphBTree(this,myNode)
2     class(BTree), intent(in) :: this
3     type(BTree), pointer, intent(in) :: myNode
4     type(BTree), pointer :: current
5     character(:),allocatable :: path
6     type(ordinal_user), pointer :: user
7     integer :: i,file
8     character(200) :: node1,casteo
9     path= "images\treeUsers.dot"
10    open(file, file=path, status="replace")
11    write(file, *) 'digraph G {'
12
13    if (.not. associated(myNode)) then
14        write(file, *) '"empty" [label="Empty papers", shape=box];'
15    else
16        write (node1,'(I13)') myNode%val(1)%DPI
17        do i = 1, myNode%num-1
18            user => myNode%val(i+1)
19            if (associated(user)) then
20                casteo=""
21                write (casteo,'(I13)') myNode%val(i+1)%DPI
22                print *, "casteo", trim(adjustl(casteo))
23                node1=trim(adjustl(node1))//",",",",",trim(adjustl(casteo))
24            end if
25        end do
26        write(file, *) " ",",", "Node", trim(adjustl(node1)), '" [label="', trim(adjustl(node1)),',"]';'
27
28        do i = 0, myNode%num
29            if (associated(myNode%link(i)%ptr)) then
30                call graphrec(myNode%link(i)%ptr,node1,file)
31            else
32                write (*,*) "no hay mas nodos"
33            end if
34        end do
35    end if
36    write(file, *) '}'
37    close(file)
38    call execute_command_line(trim("dot -Tpng images\treeUsers.dot -o images\treeUsers.png"))
39 end subroutine graphBTree

```

La subrutina graphrec es similar a graphBTree, pero se llama recursivamente en cada enlace en myNode. Toma tres argumentos: myNode (el nodo actual en el recorrido), node1 (la representación de cadena del nodo padre) y file (el identificador del archivo donde se está escribiendo el gráfico). Si myNode está asociado, se recorren los valores en myNode y se escriben en el archivo en formato de gráfico. Luego, se recorren los enlaces en myNode y se llama a graphrec en cada uno de ellos.

```

1 recursive subroutine graphrec(myNode,node1,file)
2     type(BTree), pointer, intent(in) :: myNode
3     type(BTree), pointer :: current
4     character(200), intent(inout) :: node1
5     character(200) :: node2
6     integer, intent(inout) :: file
7     !si se va a cambiar el orden hay que cambiar el limite, se estima que para grado 5 son 72
8     !pero por si las moscas mejor se le pone 200
9     character(15) :: casteo
10    integer :: i
11    if (associated(myNode)) then
12        ! do while(associated(myNode))
13        ! print *, myNode%val(1)%DPI
14        write (node2,'(I13)') myNode%val(1)%DPI
15        print *, "entro"
16        do i = 1, myNode%num-1
17            !tengo que corregir esto
18            casteo=""
19            write (casteo,'(I13)') myNode%val(i+1)%DPI
20            node2=trim(adjustl(node2))//",",",",trim(adjustl(casteo))
21        end do
22        print *, trim(adjustl(node2))
23        print *, "tamano de node", myNode%num
24        write(file, *) " ", "Node", trim(adjustl(node2)), " [label=", trim(adjustl(node2)), "];"
25
26        write(file, *) " ", "Node", trim(adjustl(node1)), " -> "Node", trim(adjustl(node2)), ";"
27        do i = 0, myNode%num
28            call graphrec(myNode%link(i)%ptr,node2,file)
29        end do
30    end if
31 end subroutine graphrec

```

La función `getUser` busca un usuario en un árbol binario de búsqueda (BTree) basándose en un DPI (Documento Personal de Identificación) y una contraseña. Esta función recibe cinco argumentos: el árbol binario (`this`), el DPI, la contraseña, un nodo del árbol (`myNode`) y un indicador lógico (`found`) que se utiliza para indicar si se ha encontrado el usuario. La función devuelve un puntero al usuario si se encuentra, y establece `found` en `.true.`. Si no se encuentra el usuario, `found` se establece en `.false.`.

```

1  function getUser(this,DPI,password,myNode,found) result(user)
2      class(BTree), intent(in) :: this
3      integer(kind=8), intent(in) :: DPI
4      character(50), intent(in) :: password
5      type(ordinal_user),pointer :: user,actualuser
6      type(BTree), pointer,intent(in) :: myNode
7      type(BTree), pointer :: myNode2
8      logical, intent(out) :: found
9      integer :: i
10     myNode2 => myNode
11     found = .false.
12     do while (.not. found)
13         if (associated(myNode2)) then
14             print *, "DPI", myNode2%val(1)%DPI, "num", myNode2%num, "password", trim(adjustl(myNode2%val(1)%password))
15             do i = 0, myNode2%num-1
16                 actualuser => myNode2%val(i+1)
17                 if (DPI<actualuser%DPI) then
18                     if (associated(myNode2%link(i)%ptr)) then
19                         myNode2 => myNode2%link(i)%ptr
20                         exit
21                     else
22                         print *, "No se encontro el usuario"
23                         return
24                     end if
25                     !si es igual al primer nodo entonces va a devolver ese nodo
26                 else if (actualuser%DPI==DPI .and. trim(adjustl(actualuser%password))==trim(adjustl(password))) then
27                     user => myNode2%val(i+1)
28                     found = .true.
29                     return
30                 end if
31             end do
32             if (DPI>myNode2%val(myNode2%num)%DPI) then
33                 myNode2 => myNode2%link(myNode2%num)%ptr
34             end if
35         else
36             print *, "No se encontro el usuario"
37             return
38         end if
39     end do
40 end function getUser

```

La función getUserAdmin es similar a getUser, pero está diseñada para ser utilizada por un administrador. No requiere una contraseña para buscar un usuario. En lugar de eso, solo necesita el DPI del usuario. Si el usuario se encuentra, la función devuelve un puntero al usuario.

```

1  function getUserAdmin(this,DPI,myNode) result(user)
2      class(BTree), intent(in) :: this
3      integer(kind=8), intent(in) :: DPI
4      type(ordinal_user),pointer :: user,actualuser
5      type(BTree), pointer,intent(in) :: myNode
6      type(BTree), pointer :: myNode2
7      logical :: found
8      integer :: i
9      myNode2 => myNode
10     found = .false.
11     user => null()
12     do while (.not. found)
13         if (associated(myNode2)) then
14             do i = 0, myNode2%num-1
15                 actualuser => myNode2%val(i+1)
16                 if (DPI<actualuser%DPI) then
17                     if (associated(myNode2%link(i)%ptr)) then
18                         myNode2 => myNode2%link(i)%ptr
19                         exit
20                     else
21                         print *, "No se encontro el usuario"
22                         return
23                     end if
24                     !si es igual al primer nodo entonces va a devolver ese nodo
25                 else if (actualuser%DPI==DPI) then
26                     user => myNode2%val(i+1)
27                     return
28                 end if
29             end do
30             if (DPI>myNode2%val(myNode2%num)%DPI) then
31                 myNode2 => myNode2%link(myNode2%num)%ptr
32             end if
33         else
34             print *, "No se encontro el usuario"
35             return
36         end if
37     end do
38 end function getUserAdmin

```

breadthFirstMatrix_adminReport, realiza un recorrido en amplitud (breadth-first traversal) del árbol binario. Esta función imprime información sobre cada usuario en el árbol, incluyendo su nombre, DPI y el número de imágenes asociadas a ese usuario. Esta función podría ser utilizada por un administrador para generar un informe sobre todos los usuarios en el sistema.



```
1  subroutine breadthFirstMatrix_adminReport(this)
2      class(BTree), intent(in) :: this
3      type(ordinal_user), pointer :: actualUser
4      type(BTree), pointer :: myNode
5      type(queue_BTree) :: queue
6      integer :: posicion,posicion2,i
7      integer(kind=8) :: DPI
8      if (.not. associated(this%root)) then
9          print *, "Arbol de usuarios esta vacio"
10         return
11     end if
12     call queue%queueaddBTree(this%root)
13     do while (associated(queue%head))
14         myNode => queue%dequeueBTree()
15         ! este es el visit()
16         do i = 0, myNode%num-1
17             actualUser => myNode%val(i+1)
18             print *, "Usuario: ", actualUser%name
19             print *, "DPI: ", actualUser%DPI
20             print *, "cantidad imagenes: ", actualUser%ImagesTree%num_images
21             print *, "-----"
22         end do
23         !
24         do i = 0, myNode%num
25             if (associated(myNode%link(i)%ptr)) then
26                 call queue%queueaddBTree(myNode%link(i)%ptr)
27             end if
28         end do
29     end do
30 end subroutine breadthFirstMatrix_adminReport
```

La subrutina queueaddBTree agrega un nodo a la cola. Esta subrutina recibe dos argumentos: la cola (this) y el nodo (myNode) que se va a agregar. Primero, se crea un nuevo nodo de cola (newNode) y se le asigna el nodo que se va a agregar. Luego, si la cola está vacía (es decir, this%head no está asociado), se establece que tanto la cabeza (head) como la cola (tail) de la cola apunten a newNode. Si la cola no está vacía, se agrega newNode al final de la cola y se actualiza this%tail para que apunte a newNode.



```
1  subroutine queueaddBTree(this,myNode)
2      class(queue_BTree), intent(inout) :: this
3      type(BTree), pointer, intent(in) :: myNode
4      type(queue_node), pointer :: newNode
5      allocate(newNode)
6      newNode%tree => myNode
7      if (.not. associated(this%head)) then
8          this%head => newNode
9          this%tail => newNode
10     else
11         this%tail%next => newNode
12         this%tail => newNode
13     end if
14 end subroutine queueaddBTree
```

La función dequeueBTree elimina un nodo de la cola y lo devuelve. Esta función recibe un argumento, la cola (this), y devuelve el nodo eliminado (myNode). Si la cola no está vacía, se establece myNode para que apunte al nodo en la cabeza de la cola, se actualiza this%head para que apunte al siguiente nodo en la cola y se desasocia el nodo de cola original (temp). Si la cola está vacía, myNode se establece en null().



```
1  function dequeueBTree(this) result(myNode)
2      class(queue_BTree), intent(inout) :: this
3      type(BTree), pointer :: myNode
4      type(queue_node), pointer :: temp
5      if (associated(this%head)) then
6          myNode => this%head%tree
7          temp => this%head
8          this%head => this%head%next
9          deallocate(temp)
10     else
11         myNode => null()
12     end if
13 end function dequeueBTree
```

Module_ordinal_user

El tipo ordinal_user tiene varios campos. Los campos name, DPI y password se utilizan para almacenar información básica del usuario. Los campos LayersTree, ImagesTree y albums son estructuras de datos que se utilizan para almacenar las capas, las imágenes y los álbumes del usuario, respectivamente.

El tipo ordinal_user también contiene varias subrutinas. Estas subrutinas proporcionan funcionalidades para visualizar reportes de estructuras, navegar y gestionar imágenes, cargar capas, imágenes y álbumes de manera masiva, y generar reportes de usuario. También hay tres subrutinas (por_recorrido_limitado, por_arbol_de_imagenes y por_capa) que están marcadas como "no usar este", lo que sugiere que estas subrutinas no se deben de utilizar solas.



```
1  use module_abbtree_layers
2  use module_jsonReader_layers
3  use module_avlTree_images
4  use module_jsonReader_images
5  use linkedListAlbum
6  use module_jsonReader_albums
7  use module_Matrix
8  use module_queue
9  implicit none
10
11  type ordinal_user
12  character(:), allocatable :: name
13  integer(kind=8), allocatable :: DPI
14  character(:), allocatable :: password
15  type(abbtree_layers) :: LayersTree
16  type(avlTree_images) :: ImagesTree
17  type(albumList) :: albums
18
19  contains
20  procedure :: ver_reportes_estructuras
21  procedure :: navegacion_gestion_imagenes
22  procedure :: carga_masiva_capas
23  procedure :: carga_masiva_imagenes
24  procedure :: carga_masiva_albumes
25  procedure :: reportes_de_usuario
26  procedure :: por_recorrido_limitado! no usar este
27  procedure :: por_arbol_de_imagenes! no usar este
28  procedure :: por_capa ! no usar este
29  end type ordinal_user
```

La subrutina `ver_reportes_estructuras` recibe un argumento, `this`, que es una instancia de la clase `ordinal_user`. Dentro de la subrutina, se define una variable lógica `found` que se utiliza para controlar el bucle `do while`. Este bucle se ejecuta hasta que `found` se establece en `.true.`. Dentro del bucle, se imprime un menú de opciones y se lee la opción seleccionada por el usuario en la variable `option`. Luego, se utiliza una estructura `select case` para realizar diferentes acciones basadas en la opción seleccionada.

```

1  subroutine ver_reportes_estructuras(this)
2      class(ordinal_user), intent(in) :: this
3      type(pixel), pointer :: actPixel
4      type(queue) :: cola
5      logical :: found
6      integer :: response,i,j,option
7      found = .false.
8      do while (.not. found)
9          print *, "-----"
10         print *, "Menu de reportes"
11         print *, "-----"
12         print *, "1. graficar arboles"
13         print *, "2. graficar matriz de una capa"
14         print *, "3. reportes del usuario"
15         print *, "4. salir"
16         read *, option
17         select case(option)
18             case(1)
19                 print *, "-----"
20                 print *, "Generando arboles"
21                 print *, "-----"
22                 call this%ImagesTree%graphImages("images") !4.3.1
23                 call this%LayersTree%graphABBTTree("layers") !4.3.2
24                 call this%albums%graphAlbumes("albumes") !4.3.3
25                 print *, "-----"
26                 print *, "arboles generados"
27                 print *, "-----"
28             case(2)
29                 call this%por_capa(acumulativo=0)!4.1.3
30             case(3)
31                 call this%reportes_de_usuario()!4.1.4
32             case(4)
33                 found = .true.
34             case default
35                 print *, "Opcion no valida"
36         end select
37     end do
38 end subroutine ver_reportes_estructuras

```

La subrutina navegacion_gestion_imagenes es similar a ver_reportes_estructuras, pero presenta un menú de opciones diferente y realiza diferentes acciones. En particular, esta subrutina permite al usuario navegar y gestionar imágenes. Si el usuario selecciona la opción

para eliminar una imagen, la subrutina busca la imagen en el árbol de imágenes y, si la encuentra, la elimina del árbol y de todos los álbumes.

```

1  subroutine navegacion_gestion_imagenes(this)
2      class(ordinal_user), intent(inout) :: this
3      type(imageList), pointer :: currentImage
4      type(album), pointer :: currentAlbum
5      class(image), pointer :: img
6      integer :: id
7      integer :: option
8      logical :: found
9      found = .false.
10     do while (.not. found)
11         print *, "-----"
12         print *, "Menu de navegacion de imagenes"
13         print *, "-----"
14         print *, "1. menu de graficacion de imagenes"
15         print *, "2. eliminar imagen"
16         print *, "3. salir"
17         print *, "-----"
18         read *, option
19         select case(option)
20             case(1)
21                 print *, "-----"
22                 print *, "menu de graficacion de imagenes"
23                 print *, "-----"
24                 print *, "1. por recorrido limitado"
25                 print *, "2. por arbol de imagenes"
26                 print *, "3. por capa"
27                 print *, "4. salir"
28                 read *, option
29                 select case(option)
30                     case(1)
31                         call this%por_recorrido_limitado()!4.1.1
32                     case(2)
33                         call this%por_arbol_de_imagenes()!4.1.2
34                     case(3)
35                         call this%por_capa(acumulativo=1)!4.1.3
36                     case(4)
37                         return
38                     case default
39                         print *, "Opcion no valida"
40                 end select
41             case(2)
42                 print *, "-----"
43                 print *, "Eliminar imagen"
44                 print *, "-----"
45                 print *, "ingrese el id de la imagen a eliminar"
46                 read *, id
47                 img => this%ImagesTree%searchImage(id)
48                 if (associated(img)) then
49                     call this%ImagesTree%deleteImg(id)
50                     call this%albums%deleteImageInAllAlbums(id)
51                     print *, "-----"
52                     print *, "Imagen eliminada"
53                     print *, "-----"
54                 else
55                     print *, "-----"
56                     print *, "Imagen no encontrada"
57                     print *, "-----"
58                 end if
59             case(3)
60                 found = .true.
61             case default
62                 print *, "Opcion no valida"
63         end select
64     end do
65 end subroutine navegacion_gestion_imagenes

```

La subrutina `carga_masiva_capas` recibe un argumento, `this`, que es una instancia de la clase `ordinal_user`. Dentro de la subrutina, se crea una instancia de `jsonReader_layers` llamada `reader`. Luego, se llama al método `readJson_Layers` de `reader`, pasando el nombre del archivo JSON y el árbol de capas del usuario como argumentos. Este método lee los datos de las capas del archivo JSON y los carga en el árbol de capas del usuario.

```
1  subroutine carga_masiva_capas(this)
2    class(ordinal_user), intent(inout) :: this
3    type(jsonReader_layers) :: reader
4    print *, "-----"
5    print *, "Carga masiva de capas"
6    print *, "-----"
7    call reader%readJson_Layers(filename="imagenmario.json",tree=this%LayersTree)
8    print *, "-----"
9    print *, "Carga masiva de capas finalizada"
10   print *, "-----"
11 end subroutine carga_masiva_capas
```

La subrutina `carga_masiva_imagenes` es similar a `carga_masiva_capas`, pero utiliza una instancia de `jsonReader_images` para leer los datos de las imágenes del archivo JSON y cargarlos en el árbol de imágenes y el árbol de capas del usuario.

```
1  subroutine carga_masiva_imagenes(this)
2    class(ordinal_user), intent(inout) :: this
3    type(jsonReader_images) :: reader
4    print *, "-----"
5    print *, "Carga masiva de imagenes"
6    print *, "-----"
7    call reader%readJson_Images(filename="img.json",abbPrincipalTree=this%LayersTree,avlPrincipalTree=this%ImagesTree)
8    print *, "-----"
9    print *, "Carga masiva de imagenes finalizada"
10   print *, "-----"
11 end subroutine carga_masiva_imagenes
```

La subrutina `carga_masiva_albumes` también es similar, pero utiliza una instancia de `jsonReader_albumsJson` para leer los datos de los álbumes del archivo JSON y cargarlos en la lista de álbumes del usuario.

```

1  subroutine carga_masiva_albumes(this)
2      class(ordinal_user), intent(inout) :: this
3      type(jsonReader_albumsJson) :: reader
4      print *, "-----"
5      print *, "Carga masiva de albumes"
6      print *, "-----"
7      call reader%readJson_albumsJson(filename="albumes.json",albums=this%albums)
8      print *, "-----"
9      print *, "Carga masiva de albumes finalizada"
10     print *, "-----"
11 end subroutine carga_masiva_albumes

```

La subrutina por_recorrido_limitado permite al usuario seleccionar una imagen por su ID y luego graficar un número específico de capas de esa imagen. Primero, se limpia una cola y se llena con los IDs de las imágenes en orden. Luego, el usuario ingresa el ID de la imagen que desea graficar. Si la imagen existe, se le pide al usuario que ingrese el número de capas que desea graficar. Luego, el usuario puede seleccionar el tipo de recorrido (preorden, inorden, postorden) para recorrer el árbol de capas. Para cada capa seleccionada, se extraen los píxeles y se insertan en una matriz, que luego se grafica.

```

1  subroutine por_recorrido_limitado(this)
2      class(ordinal_user), intent(in) :: this
3      type(queue) :: cola
4      type(abbtree_layers) :: tree
5      type(pixellist) :: pixels
6      type(matrix) :: actmatrix
7      type(pixel), pointer :: actPixel
8      class(image), pointer :: img
9      type(layer), pointer :: layertemp
10     character(:), allocatable :: recorrido
11     character(15) :: casteo
12     integer :: id,num,i
13     logical :: salir
14     salir = .false.
15     actmatrix%root => null()
16     print *, "-----"
17     print *, "Menu de recorrido limitado"
18     print *, "-----"
19     print *, "ingrese el id de la imagen a graficar"
20     call cola%cleanQueue()
21     call this%ImagesTree%inorderAVL(this%ImagesTree%root,cola)
22     call cola%printQueue()
23     print *, "-----"
24     read *, id
25     img => this%ImagesTree%searchImage(id)
26     if (associated(img)) then
27         tree = img%abb
28         print *, "ingrese el numero de capas a graficar"
29         print *, "si el numero es mayor a la cantidad se graficara todo"
30         read *, num
31         do while (.not. salir)
32             print *, "-----tipo de orden-----"
33             print *, "1. preorden"
34             print *, "2. inorden"
35             print *, "3. postorden"
36             print *, "4. salir"
37             read *, id
38             select case(id)
39                 case(1)
40                     call cola%cleanQueue()
41                     call tree%preorderABB(tree%root,cola)
42                     recorrido = "preorden: "
43                 case(2)
44                     call cola%cleanQueue()
45                     call tree%inorderABB(tree%root,cola)
46                     recorrido = "inorden: "
47                 case(3)
48                     call cola%cleanQueue()
49                     call tree%postorderABB(tree%root,cola)
50                     recorrido = "postorden: "
51                 case(4)
52                     salir = .true.
53                     return
54                 case default
55                     print *, "Opcion no valida"
56             end select
57             do i = 1, num
58                 if (.not.colaisEmpty()) then
59                     id = cola%dequeue()
60                     layertemp => tree%searchLayer(id)
61                     actPixel => layertemp%pixels%head
62                     write(casteo,'(I15)') id
63                     if (i==num) then
64                         recorrido = trim(adjustl(recorrido))// " " // trim(adjustl(casteo))
65                     else
66                         recorrido = trim(adjustl(recorrido))// " " // trim(adjustl(casteo))// ","
67                     end if
68                     do while (associated(actPixel))
69                         call actmatrix%insert(i=actPixel%row,j=actPixel%col,color=actPixel%color)
70                         actPixel => actPixel%next
71                     end do
69                     else
72                         exit
73                     end if
74                 end do
75                 call actmatrix%graphTable("img_recorrido_limitado",trim(adjustl(recorrido)))
76             end do
77         end if
78     end subroutine por_recorrido_limitado

```

La subrutina `por_arbol_de_imagenes` permite al usuario seleccionar una imagen por su ID y luego graficar la imagen completa. Al igual que en la subrutina anterior, se llena una cola con los IDs de las imágenes en orden. Luego, el usuario ingresa el ID de la imagen que desea graficar. Si la imagen existe, se grafica la imagen completa. Si no, se informa al usuario que la imagen no se encontró.

```
1  subroutine por_arbol_de_imagenes(this)
2      class(ordinal_user), intent(in) :: this
3      type(matrix) :: actmatrix
4      type(queue) :: cola
5      class(image), pointer :: img
6      integer :: response
7      logical :: found
8      found = .false.
9      actmatrix%root => null()
10     print *, "-----"
11     print *, "que imagen desea graficar? (ingresar id de la imagen)"
12     call cola%cleanQueue()
13     call this%ImagesTree%inorderAVL(this%ImagesTree%root,cola)
14     call cola%printQueue()
15     print *, "-----"
16     print *, "arriba estan las imagenes disponibles (ingrese '-1' para salir)"
17     print *, "-----"
18     read *, response
19     if (response /= -1) then
20         img => this%ImagesTree%searchImage(response)
21         if (associated(img)) then
22             call this%ImagesTree%breadthFirstMatrix(actualMatrix=actmatrix,idImage=response)
23             call actmatrix%graphMatrix("matrix_image")
24             call actmatrix%graphTable("image"," ")
25             call actmatrix%cleanMatrix()
26             print *, "-----"
27             print *, "Imagen graficada"
28             print *, "-----"
29         else
30             print *, "-----"
31             print *, "Imagen no encontrada"
32             print *, "-----"
33         end if
34     else
35         return
36     end if
37 end subroutine por_arbol_de_imagenes
```

`por\_capa`, permite al usuario seleccionar una capa específica para graficar. La subrutina toma dos argumentos: una instancia de la clase `ordinal\_user` y un entero `acumulativo`. La subrutina comienza inicializando una matriz, una cola y varios punteros y variables. Luego, entra en un bucle que continúa hasta que se encuentra la capa deseada.

Dentro del bucle, si el árbol de capas del usuario tiene un nodo raíz asociado, se le pide al usuario que ingrese el ID de la capa que desea graficar. Luego, se limpia la cola y se llena con los IDs de las capas en orden. Si el usuario ingresa un ID que no es -1, se busca la capa correspondiente en el árbol de capas y se recorren sus píxeles. Cada pixel se inserta en la

matriz y, una vez que se han recorrido todos los píxeles, se grafica la matriz. Si el argumento `acumulativo` no es 1, se limpia la matriz. Si el usuario ingresa -1, la subrutina termina. Si no hay capas en el árbol de capas, se informa al usuario y se sale del bucle.

```
1  subroutine por_capa(this,acumulativo)
2      class(ordinal_user), intent(in) :: this
3      integer, intent(in) :: acumulativo
4      type(matrix) :: actmatrix
5      type(queue) :: cola
6      type(layer), pointer :: layertemp
7      type(pixel), pointer :: actPixel
8      integer :: response
9      logical :: found
10     integer :: i,j
11     found = .false.
12     actmatrix%root => null()
13     do while (.not. found) !4.3.4
14         if (associated(this%LayersTree%root)) then
15             print *, "-----"
16             print *, "que capa desea graficar? (ingresar id de la capa)"
17             call cola%cleanQueue()
18             call this%LayersTree%inorderABB(this%LayersTree%root,cola)
19             call cola%printQueue()
20             print *, "-----"
21             print *, "arriba estan las capas disponibles (ingrese '-1' para salir)"
22             print *, "-----"
23             read *, response
24             if (response /= -1) then
25                 layertemp => this%LayersTree%searchLayer(response)
26                 actPixel => layertemp%pixels%head
27                 do while (associated(actPixel))
28                     i = actPixel%row
29                     j = actPixel%col
30                     call actmatrix%insert(i=i,j=j,color=actPixel%color)
31                     actPixel => actPixel%next
32                 end do
33                 call actmatrix%graphTable(filename="matrix_layer",text=" ")
34                 if (acumulativo /= 1) then
35                     call actmatrix%cleanMatrix()
36                 end if
37             else
38                 return
39             end if
40         else
41             print *, "No hay capas"
42             exit
43         end if
44     end do
45 end subroutine por_capa
```

`reportes\_de\_usuario`, genera varios informes para el usuario. La subrutina toma una instancia de la clase `ordinal\_user` como argumento. Comienza imprimiendo información básica sobre el usuario, como su nombre, DPI y contraseña. Luego, genera un informe de las cinco imágenes con más capas, seguido de un informe de todas las capas que son hojas en el árbol de capas. Finalmente, imprime la profundidad del árbol de capas y realiza recorridos en preorden, inorden y postorden del árbol de capas, imprimiendo los IDs de las capas en cada recorrido.

```
1  subroutine reportes_de_usuario(this)
2      class(ordinal_user), intent(in) :: this
3      type(queue) :: cola
4      integer :: num_images
5      print *, "-----"
6      print *, "Reportes de usuario"
7      print *, "-----"
8      print *, "usuario: ", this%name
9      print *, "DPI: ", this%DPI
10     print *, "password: ", this%password
11     print *, "-----"
12     print *, "top 5 imagenes con mas numeros de capas"
13     call this%ImagesTree%inorderAVL(this%ImagesTree%root,cola)
14     num_images = cola%getQueueSize()
15     call this%ImagesTree%top_five_images_with_more_layers(cola,num_images)
16     call cola%cleanQueue()
17     print *, "-----"
18     print *, "todas las capas que son hojas"
19     call this%LayersTree%leaf_layers(this%LayersTree%root)
20     print *, ""
21     print *, "-----"
22     print *, "profundidad de arbol de capas: ", this%LayersTree%profundidad
23     print *, "-----capas-----"
24     call this%LayersTree%preorderABB(this%LayersTree%root,cola)
25     write(*,"(A)", advance='no') "preorden"
26     call cola%printQueue()
27     call cola%cleanQueue()
28     call this%LayersTree%inorderABB(this%LayersTree%root,cola)
29     write(*,"(A)", advance='no') "inorden"
30     call cola%printQueue()
31     call cola%cleanQueue()
32     call this%LayersTree%postorderABB(this%LayersTree%root,cola)
33     write(*,"(A)", advance='no') "postorden"
34     call cola%printQueue()
35     print *, "-----"
36 end subroutine reportes_de_usuario
```

linkedListAlbum

El tipo album tiene cuatro campos: name que es una cadena de caracteres, [images](#) que es una lista de imágenes, y dos punteros next y prev que apuntan al siguiente y al anterior álbum en la lista, respectivamente.



```
1  type album
2      character(:), allocatable :: name
3      type(imageList) :: images
4      type(album), pointer :: next => null()
5      type(album), pointer :: prev => null()
6  end type
```

El tipo albumList tiene dos punteros, head y tail, que apuntan al primer y último álbum de la lista, respectivamente. También contiene una serie de procedimientos que permiten manipular la lista de álbumes y las imágenes dentro de cada álbum.



```
1  type albumList
2      type(album), pointer :: head => null()
3      type(album), pointer :: tail => null()
4      contains
5      procedure :: addAlbum
6      procedure :: searchAlbum
7      procedure :: graphAlbumes
8      procedure :: addImageInAlbum
9      procedure :: deleteImageInAlbum
10     procedure :: deleteImageInAllAlbums
11     procedure :: cuantitiImagesInAlbums
12     procedure :: cuantitiAlbums
13 end type
```

El procedimiento addAlbum agrega un nuevo álbum a la lista. Primero busca si el álbum ya existe en la lista. Si no existe, crea un nuevo álbum y lo agrega al final de la lista.



```
1  subroutine addAlbum(this,name)
2      class(albumList), intent(inout) :: this
3      character(len=*), intent(in) :: name
4      type(album), pointer :: newAlbum,test
5      test => this%searchAlbum(name)
6      if (.not. associated(test)) then
7          allocate(newAlbum)
8          newAlbum%name = name
9          if (.not. associated(this%head)) then
10             this%head => newAlbum
11             this%tail => newAlbum
12         else
13             this%tail%next => newAlbum
14             newAlbum%prev => this%tail
15             this%tail => newAlbum
16         end if
17     end if
18 end subroutine addAlbum
```

El procedimiento searchAlbum busca un álbum en la lista por su nombre. Recorre la lista desde el principio hasta el final, y si encuentra un álbum con el nombre buscado, devuelve un puntero a ese álbum.



```
1  function searchAlbum(this,name) result(res)
2      class(albumList), intent(in) :: this
3      character(len=*), intent(in) :: name
4      type(album), pointer :: current
5      type(album), pointer :: res
6      current => this%head
7      res => null()
8      do while (associated(current))
9          if (current%name == name) then
10             print *, "Album found"
11             res => current
12             return
13         end if
14         current => current%next
15     end do
16     print *, "Album not found"
17 end function searchAlbum
```

El procedimiento graphAlbumes genera un gráfico de la lista de álbumes y las imágenes dentro de cada álbum. Este gráfico se guarda en un archivo con formato DOT, que luego se convierte en una imagen PNG.

```

1  subroutine graphAlbumes(this,filename)
2      class(albumList), intent(in) :: this
3      character(len=*) , intent(in) :: filename
4      type(album), pointer :: current
5      type(image), pointer :: currentImage
6      character(:), allocatable :: path,rank
7      character(:), allocatable :: conexion
8      integer :: file
9      path = "images\\"//trim(adjustl(filename))//".dot"
10     open(file, file=path, status="replace")
11     write(file, *) 'digraph G {'
12     write(file, *) 'node [shape=box];'
13     current => this%head
14     if (associated(current)) then
15         rank = "{rank=same"
16         do while (associated(current))
17             write(file, *) "Album", trim(adjustl(current%name)), " [label='', trim(adjustl(current%name)),'"];'"
18             rank = trim(adjustl(rank))//";"Album'//trim(adjustl(current%name))//'"
19             currentImage => current%images%head
20             if (associated(currentImage)) then
21                 write(file, *) "Image", currentImage%id, " [label='', currentImage%id,'"];'"
22                 write(file, *) "Album", trim(adjustl(current%name)), " -> "Image', currentImage%id, ",'"
23                 ! rank = trim(adjustl(rank))//'"Album"//trim(adjustl(current%name))//'"
24                 do while (associated(currentImage%next))
25                     write(file, *) "Image", currentImage%next%id, " [label='', currentImage%next%id,'"];'"
26                     write(file, *) "Image", currentImage%id, " -> "Image', currentImage%next%id, ",'"
27                     currentImage => currentImage%next
28                 end do
29             else
30                 print *, "No images in album"
31             end if
32             if (associated(current%next)) then
33                 write(file, *) "Album", trim(adjustl(current%name)), " [label='', trim(adjustl(current%name)),'"];'"
34                 conexion = "Album'// trim(adjustl(current%name))// ' " -> "Album'// trim(adjustl(current%next%name))
35                 write(file, *) conexion// " [color=blue];'"
36                 write(file, *) conexion// " [color=red][dir = back];'"
37             end if
38             current => current%next
39         end do
40         write(file, *) rank, "}"
41     else
42         write(file, *) "empty" [label="Empty albumes", shape=box];'"
43     end if
44     write(file, *) '}'
45     close(file)
46     call execute_command_line("dot -Tpng images\albumes.dot -o images\albumes.png")
47     ! windows
48     call system("start images\\"//trim(adjustl(filename))//".png")
49     ! linux
50     ! call system("xdg-open images\\"//trim(adjustl(filename))//".png")
51 end subroutine graphAlbumes

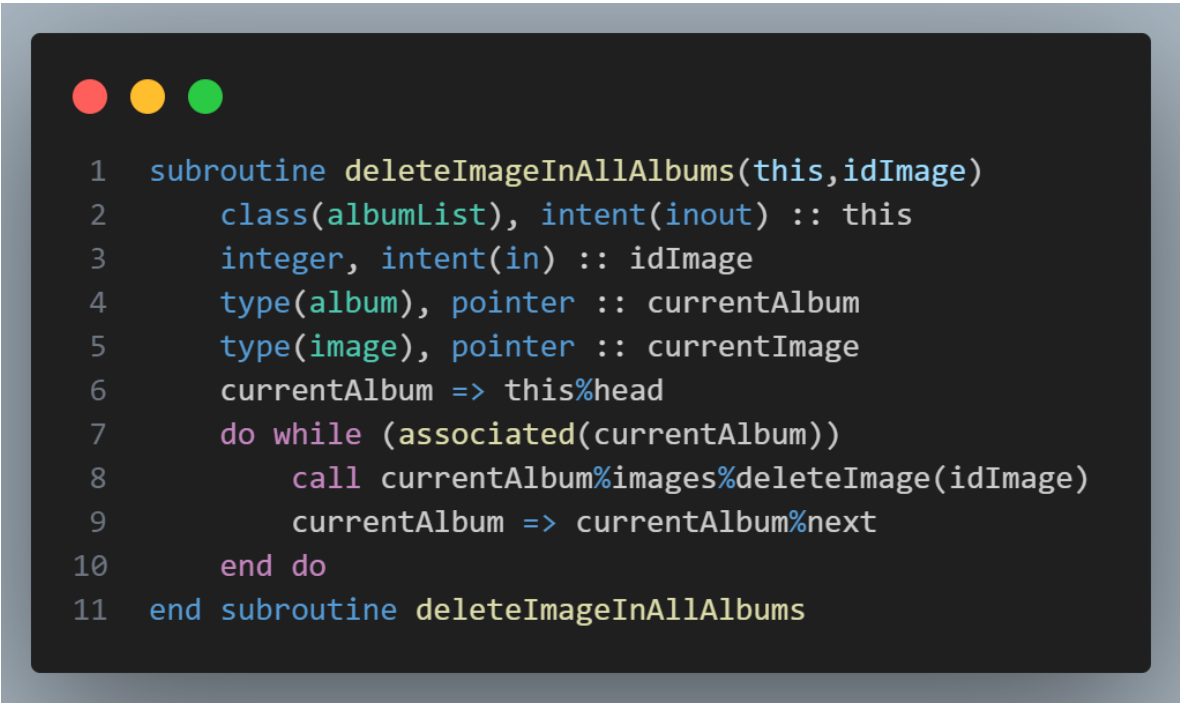
```

Los procedimientos addImageInAlbum y deleteImageInAlbum permiten agregar y eliminar imágenes en un álbum específico, respectivamente.



```
1  subroutine addImageInAlbum(this,nameAlbum,idImage)
2      class(albumList), intent(inout) :: this
3      character(len=*), intent(in) :: nameAlbum
4      integer, intent(in) :: idImage
5      type(album), pointer :: currentAlbum
6      type(image), pointer :: newImage
7      currentAlbum => this%searchAlbum(nameAlbum)
8      if (associated(currentAlbum)) then
9          call currentAlbum%images%addImage(idImage)
10     else
11         print *, "Album not found"
12     end if
13 end subroutine addImageInAlbum
14
15 subroutine deleteImageInAlbum(this,nameAlbum,idImage)
16     class(albumList), intent(inout) :: this
17     character(len=*), intent(in) :: nameAlbum
18     integer, intent(in) :: idImage
19     type(album), pointer :: currentAlbum
20     currentAlbum => this%searchAlbum(nameAlbum)
21     if (associated(currentAlbum)) then
22         call currentAlbum%images%deleteImage(idImage)
23     else
24         print *, "Album not found"
25     end if
26 end subroutine deleteImageInAlbum
```

El procedimiento deleteImageInAllAlbums elimina una imagen de todos los álbumes en la lista.



```
1  subroutine deleteImageInAllAlbums(this,idImage)
2      class(albumList), intent(inout) :: this
3      integer, intent(in) :: idImage
4      type(album), pointer :: currentAlbum
5      type(image), pointer :: currentImage
6      currentAlbum => this%head
7      do while (associated(currentAlbum))
8          call currentAlbum%images%deleteImage(idImage)
9          currentAlbum => currentAlbum%next
10     end do
11 end subroutine deleteImageInAllAlbums
```

Las funciones `cuantitiImagesInAlbums` y `cuantitiAlbums` devuelven el número total de imágenes en todos los álbumes y el número total de álbumes en la lista, respectivamente.



```
1  function cuantitiImagesInAlbums(this) result(counter)
2      class(albumList), intent(in) :: this
3      type(album), pointer :: currentAlbum
4      type(image), pointer :: currentImage
5      integer :: counter
6      counter = 0
7      currentAlbum => this%head
8      do while (associated(currentAlbum))
9          currentImage => currentAlbum%images%head
10         do while (associated(currentImage))
11             counter = counter + 1
12             currentImage => currentImage%next
13         end do
14         currentAlbum => currentAlbum%next
15     end do
16 end function cuantitiImagesInAlbums
17
18 function cuantitiAlbums(this) result(counter)
19     class(albumList), intent(in) :: this
20     type(album), pointer :: currentAlbum
21     integer :: counter
22     counter = 0
23     currentAlbum => this%head
24     do while (associated(currentAlbum))
25         counter = counter + 1
26         currentAlbum => currentAlbum%next
27     end do
28 end function cuantitiAlbums
```

linkedListImages

El módulo define dos tipos de datos: `image` e `imageList`.

El tipo `image` tiene dos campos: `id` que es un entero, y `next` que es un puntero al siguiente nodo en la lista.



```
1  type image
2      integer :: id
3      type(image), pointer :: next => null()
4  end type
```

El tipo `imageList` tiene dos punteros, `head` y `tail`, que apuntan al primer y último nodo de la lista, respectivamente. También contiene una serie de procedimientos que permiten manipular la lista de imágenes.



```
1  type imageList
2      type(image), pointer :: head => null()
3      type(image), pointer :: tail => null()
4  contains
5      procedure :: addImage
6      procedure :: deleteImage
7      procedure :: existID
8      procedure :: searchid
9  end type
```

El procedimiento `addImage` agrega una nueva imagen a la lista. Primero busca si la imagen ya existe en la lista. Si no existe, crea una nueva imagen y la agrega al final de la lista. Si la imagen ya existe, simplemente imprime un mensaje y actualiza el `id` de la imagen existente.



```
1  subroutine addImage(this,id)
2      class(imageList), intent(inout) :: this
3      integer, intent(in) :: id
4      type(image), pointer :: newImage,test
5      test => this%searchid(id)
6      if (.not. associated(test)) then
7          allocate(newImage)
8          newImage%id = id
9          if (.not. associated(this%head)) then
10             this%head => newImage
11             this%tail => newImage
12         else
13             this%tail%next => newImage
14             this%tail => newImage
15         end if
16     else
17         print *, "Image already exists"
18         test%id = id
19     end if
20 end subroutine addImage
```

El procedimiento `deleteImage` elimina una imagen de la lista. Recorre la lista desde el principio hasta el final, y si encuentra una imagen con el `id` buscado, la elimina de la lista.



```
1  subroutine deleteImage(this,id)
2      class(imageList), intent(inout) :: this
3      integer, intent(in) :: id
4      type(image), pointer :: current
5      type(image), pointer :: previous
6      current => this%head
7      previous => null()
8      do while (associated(current))
9          if (current%id == id) then
10             if (associated(previous)) then
11                 previous%next => current%next
12             else
13                 this%head => current%next
14             end if
15             deallocate(current)
16             exit
17         end if
18         previous => current
19         current => current%next
20     end do
21 end subroutine deleteImage
```

El procedimiento `existID` verifica si una imagen existe en la lista. Recorre la lista desde el principio hasta el final, y si encuentra una imagen con el `id` buscado, imprime un mensaje y termina la búsqueda.



```
1  subroutine existID(this,id)
2      class(imageList), intent(in) :: this
3      integer, intent(in) :: id
4      type(image), pointer :: current
5      current => this%head
6      do while (associated(current))
7          if (current%id == id) then
8              print *, "Image found"
9              exit
10         end if
11         current => current%next
12     end do
13     print *, "Image not found"
14 end subroutine existID
```

La función `searchid` busca una imagen en la lista por su `id`. Recorre la lista desde el principio hasta el final, y si encuentra una imagen con el `id` buscado, devuelve un puntero a esa imagen. Si no encuentra la imagen, imprime un mensaje y devuelve un puntero nulo.



```
1  function searchid(this,id) result(res)
2      class(imageList), intent(inout) :: this
3      integer, intent(in) :: id
4      type(image), pointer :: current
5      type(image), pointer :: res
6      current => this%head
7      res => null()
8      do while (associated(current))
9          if (current%id == id) then
10             print *, "Image found"
11             res => current
12             return
13         end if
14         current => current%next
15     end do
16     print *, "Image not found"
17 end function searchid
```

Module_admin_user

Dentro del módulo, se define un tipo de datos llamado admin_user. Este tipo de datos tiene varios campos, incluyendo name, DPI, password y treeUsers. treeUsers es un árbol B que se utiliza para almacenar usuarios. Además, el tipo admin_user tiene varios procedimientos asociados, que permiten manipular los datos de los usuarios.



```
1  type admin_user
2  character(:), allocatable :: name
3  integer(kind=8), allocatable :: DPI
4  character(:), allocatable :: password
5  type(BTree) :: treeUsers
6  contains
7  procedure :: arbolB_de_usuarios
8  procedure :: insertar_usuario
9  procedure :: eliminar_usuario
10 procedure :: modificar_usuario
11 procedure :: carga_masiva_usuarios
12 procedure :: reportes_de_admin_user
13 end type
```

El procedimiento `arbolB_de_usuarios` crea un árbol B de usuarios. Si el árbol se crea con éxito, imprime un mensaje indicando que el árbol de usuarios se ha creado. Si no, imprime un mensaje indicando que no se pudo crear el árbol de usuarios.

```

1  subroutine arbolB_de_usuarios(this)
2      class(admin_user), intent(inout) :: this
3      if (associated(this%treeUsers%returnRoot())) then
4          print *, "-----"
5          print *, "haciendo arbol de usuarios"
6          print *, "-----"
7          call this%treeUsers%graphBTree(myNode=this%treeUsers%returnRoot())
8          print *, "-----"
9          print *, "Arbol de usuarios hecho"
10         print *, "-----"
11     else
12         print *, "-----"
13         print *, "No se ha podido hacer el arbol de usuarios"
14         print *, "-----"
15     end if
16 end subroutine

```

El procedimiento insertar_usuario permite insertar un nuevo usuario en el árbol B de usuarios. Solicita al usuario que ingrese el nombre, el DPI y la contraseña del nuevo usuario. Si el DPI es válido, crea un nuevo usuario y lo inserta en el árbol.

```

1  subroutine insertar_usuario(this)
2      class(admin_user), intent(inout) :: this
3      type(ordinal_user) :: user
4      character(50) :: nombre
5      integer(kind=8) :: DPI
6      character(50) :: password
7      print *, "-----"
8      print *, "Ingrese el nombre del usuario"
9      read *, nombre
10     print *, "Ingrese el DPI del usuario (numeros)"
11     read *, DPI
12     print *, "Ingrese la contraseña del usuario"
13     read *, password
14     if (DPI<9999999999999999_8 .and. DPI>9999999999999999_8) then
15         user = ordinal_user(nombre, DPI, password)
16         call this%treeUsers%insert(user)
17         print *, "-----"
18         print *, "Usuario insertado"
19         print *, "-----"
20     else
21         print *, "-----"
22         print *, "DPI no valido"
23         print *, "-----"
24     end if
25 end subroutine

```

El procedimiento eliminar_usuario permite eliminar un usuario del árbol B de usuarios. Solicita al usuario que ingrese el DPI del usuario a eliminar. Si el usuario existe, lo elimina del árbol.

```

1  subroutine eliminar_usuario(this)
2      class(admin_user), intent(inout) :: this
3      type(ordinal_user), pointer :: user
4      integer(kind=8) :: DPI
5      print *, "-----"
6      print *, "Ingrese el DPI del usuario a eliminar (numeros)"
7      read *, DPI
8      user => this%treeUsers%getUserAdmin(DPI=DPI, myNode=this%treeUsers%root)
9      if (associated(user)) then
10         call this%treeUsers%remove(DPI)
11         print *, "-----"
12         print *, "Usuario eliminado"
13         print *, "-----"
14     else
15         print *, "-----"
16         print *, "Usuario no encontrado"
17         print *, "-----"
18     end if
19 end subroutine

```

El procedimiento `modificar_usuario` permite modificar los datos de un usuario existente en el árbol B de usuarios. Solicita al usuario que ingrese el DPI y la contraseña del usuario a modificar. Si el usuario existe, permite modificar su nombre, DPI y contraseña.

El procedimiento `carga_masiva_usuarios` permite cargar usuarios desde un archivo JSON llamado "users.json".



```

1  subroutine carga_masiva_usuarios(this)
2      class(admin_user), intent(inout) :: this
3      type(jsonReader) :: json
4      character(:), allocatable :: filename
5      print *, "-----"
6      print *, "cargando usuarios"
7      print *, "-----"
8
9      call json%readJson("users.json", this%treeUsers)
10
11     print *, "-----"
12     print *, "usuarios cargados"
13     print *, "-----"
14
15 end subroutine

```

Finalmente, el procedimiento `reportes_de_admin_user` permite generar varios informes sobre los usuarios. Puede mostrar información sobre un usuario específico, listar todos los usuarios en un recorrido de nivel o salir del procedimiento.

Module_layer

El tipo `pixel` representa un píxel individual en una imagen. Tiene cuatro campos: `row` y `col`, que representan la posición del píxel en la imagen; `color`, que representa el color del píxel; y `next`, que es un puntero al siguiente píxel en una lista de píxeles.



```
1  type :: pixel
2      integer :: row
3      integer :: col
4      character(len=7) :: color
5      type(pixel), pointer :: next => null()
6  end type pixel
```

El tipo `pixelList` representa una lista de píxeles. Tiene dos campos: `head` y `tail`, que son punteros al primer y último píxel de la lista, respectivamente. También tiene dos procedimientos asociados: `insertPixel` y `getHead`. `insertPixel` permite insertar un nuevo píxel en la lista. `getHead` devuelve el primer píxel de la lista.



```
1  type :: pixelList
2      type(pixel), pointer :: head => null()
3      type(pixel), pointer :: tail => null()
4      contains
5          procedure :: insertPixel
6          procedure :: getHead
7  end type pixelList
```

El tipo `layer` representa una capa de una imagen. Tiene cuatro campos: `id`, que es un identificador único para la capa; `pixels`, que es una lista de píxeles en la capa; y `right` y `left`, que son punteros a las capas adyacentes.



```
1  type :: layer
2      integer :: id
3      type(pixelList) :: pixels
4      type(layer), pointer :: right => null()
5      type(layer), pointer :: left => null()
6  end type layer
```

La subrutina insertPixel toma cuatro argumentos: una referencia a una lista de píxeles, y los valores de fila, columna y color para un nuevo píxel. Crea un nuevo píxel con estos valores y lo inserta al final de la lista. Si la lista está vacía, el nuevo píxel se convierte en el primer y último píxel de la lista.



```
1  subroutine insertPixel(this, row, col, color)
2      class(pixelList), intent(inout) :: this
3      integer, intent(in) :: row, col
4      character(len=7), intent(in) :: color
5      type(pixel), pointer :: temp
6      allocate(temp)
7      temp = pixel(col=col, row=row, color=color)
8      if (.not. associated(this%head)) then
9          this%head => temp
10         this%tail => temp
11     else
12         this%tail%next => temp
13         this%tail => this%tail%next
14     end if
15 end subroutine insertPixel
```

La función getHead toma una referencia a una lista de píxeles y devuelve el primer píxel de la lista.



```
1  function getHead(this) result(head)
2      class(pixelList), intent(in) :: this
3      type(pixel), pointer :: head
4      head => this%head
5  end function getHead
```

Module_abbtree_layers

El módulo define un tipo de datos `abbtree\_layers` que contiene un puntero a la raíz del árbol (`root`), la profundidad del árbol (`profundidad`), y el número de capas en el árbol (`num\_layers`). También define varios procedimientos para manipular el árbol, incluyendo `insertLayer`, `deleteLayer`, `preorderABB`, `inorderABB`, `postorderABB`, `graphABBTre`, `searchLayer`, `searchLayer\_Rec`, `insertRec`, y `leaf\_layers`.

```
1  type :: abbtree_layers
2      type(layer), pointer :: root => null()
3      integer :: profundidad = 0
4      integer :: num_layers = 0
5  contains
6      procedure :: insertLayer
7      procedure :: deleteLayer
8      procedure :: preorderABB
9      procedure :: inorderABB
10     procedure :: postorderABB
11     procedure :: graphABBTre
12     procedure :: searchLayer
13     procedure :: searchLayer_Rec
14     procedure :: insertRec
15     procedure :: leaf_layers
16 end type abbtree_layers
```

El procedimiento `insertLayer` inserta una nueva capa en el árbol. Si el árbol está vacío, la nueva capa se convierte en la raíz. Si el árbol no está vacío, se llama al procedimiento `insertRec` para insertar la capa en la posición correcta en el árbol.



```
1  subroutine insertLayer(this, id, pixels)
2      class(abbtrees_layers), intent(inout) :: this
3      integer, intent(in) :: id
4      type(pixelList), intent(in) :: pixels
5      type(layer), pointer :: capa
6      integer :: profundidad
7      profundidad = 0
8      allocate(capa)
9      capa = layer(id=id, pixels=pixels)
10     if (.not. associated(this%root)) then
11         this%root => capa
12         this%num_layers = this%num_layers + 1
13     else
14         call this%insertRec(this%root, capa, profundidad)
15         if (profundidad > this%profundidad) then
16             this%profundidad = profundidad
17         end if
18     end if
19 end subroutine insertLayer
20
21 recursive subroutine insertRec(this, root, capa, profundidad)
22     class(abbtrees_layers), intent(inout) :: this
23     type(layer), pointer, intent(inout) :: root
24     class(layer), pointer, intent(in) :: capa
25     integer, intent(inout) :: profundidad
26     ! print *, "capa: ", capa%id, "capa2: ", root%id
27     if (capa%id < root%id) then
28         profundidad = profundidad + 1
29         if (.not. associated(root%left)) then
30             this%num_layers = this%num_layers + 1
31             allocate(root%left)
32             root%left => capa
33         else
34             call this%insertRec(root%left, capa, profundidad)
35         end if
36     else if (capa%id > root%id) then
37         profundidad = profundidad + 1
38         if (.not. associated(root%right)) then
39             this%num_layers = this%num_layers + 1
40             allocate(root%right)
41             root%right => capa
42         else
43             call this%insertRec(root%right, capa, profundidad)
44         end if
45     else if (capa%id == root%id) then
46         !revisar esto
47         root => capa
48     end if
49 end subroutine insertRec
```

El procedimiento `deleteLayer` elimina una capa del árbol. Llama a la función `deleteRec` para encontrar la capa y eliminarla.



```

1  subroutine deleteLayer(this, capa)
2      class(abbtrees_layers), intent(inout) :: this
3      integer, intent(in) :: capa
4
5      this%root => deleteRec(this%root, capa)
6  end subroutine deleteLayer
7
8  !no usar por separado
9  recursive function deleteRec(root, key) result(res)
10     type(layer), pointer :: root
11     integer, intent(in) :: key
12     type(layer), pointer :: res
13     type(layer), pointer :: temp
14
15     if (.not. associated(root)) then
16         res => root
17         return
18     end if
19
20     if (key < root%id) then
21         root%left => deleteRec(root%left, key)
22     else if (key > root%id) then
23         root%right => deleteRec(root%right, key)
24     else
25         if (.not. associated(root%left)) then
26             temp => root%right
27             deallocate(root)
28             res => temp
29             return
30         else if (.not. associated(root%right)) then
31             temp => root%left
32             deallocate(root)
33             res => temp
34             return
35         else
36             call getMajorOfMinorsLayers(root%left, temp)
37             root%id = temp%id
38             root%left => deleteRec(root%left, temp%id)
39         end if
40     end if
41
42     res => root
43 end function deleteRec

```

Los procedimientos ``preorderABB``, ``inorderABB``, y ``postorderABB`` realizan recorridos en preorden, inorden y postorden del árbol, respectivamente. Cada uno de estos procedimientos utiliza una cola para almacenar los identificadores de las capas a medida que se visitan.



```
1  subroutine preorderABB(this, tmp,cola)
2      class(abbtrees_layers), intent(in) :: this
3      type(layer), intent(in), pointer :: tmp
4      type(Queue), intent(inout) :: cola
5      if(associated(tmp)) then
6          call cola%enqueue(tmp%id)
7          call this%preorderABB(tmp%left,cola)
8          call this%preorderABB(tmp%right,cola)
9      else
10         return
11     end if
12 end subroutine preorderABB
13
14 subroutine inorderABB(this, tmp,cola)
15     class(abbtrees_layers), intent(in) :: this
16     type(layer), intent(in), pointer :: tmp
17     type(Queue), intent(inout) :: cola
18     if(associated(tmp)) then
19         call this%inorderABB(tmp%left,cola)
20         call cola%enqueue(tmp%id)
21         call this%inorderABB(tmp%right,cola)
22     else
23         return
24     end if
25 end subroutine inorderABB
26
27 subroutine postorderABB(this, tmp,cola)
28     class(abbtrees_layers), intent(in) :: this
29     type(layer), intent(in), pointer :: tmp
30     type(Queue), intent(inout) :: cola
31     if(associated(tmp)) then
32         call this%postorderABB(tmp%left,cola)
33         call this%postorderABB(tmp%right,cola)
34         call cola%enqueue(tmp%id)
35     else
36         return
37     end if
38 end subroutine postorderABB
```

El procedimiento `graphABBTre` genera un gráfico del árbol utilizando el software Graphviz. Crea un archivo .dot, lo convierte en una imagen .png y luego abre la imagen.

```
1  subroutine graphABBTre(this, filename)
2      class(abbtrec_layers), intent(in) :: this
3      character(len=*), intent(in) :: filename
4      character(:),allocatable :: path
5      integer :: file
6      path = "images/"//trim(adjustl(filename))//".dot"
7      open(file, file=path, status="replace")
8      write(file, '(A)') 'digraph{'
9      if (associated(this%root)) then
10         call graphABBTre_rec(this%root, file)
11     else
12         write(file, '(A)') '"empty" [label="Empty layers", shape=box];'
13     end if
14     write(file, '(A)') '}'
15     close(file)
16     call execute_command_line("dot -Tpng images/"//trim(adjustl(filename))//".dot -o images/"//trim(adjustl(filename))//".png")
17     !windows
18     call system("start images\\"//trim(adjustl(filename))//".png")
19     ! linux
20     ! call system("xdg-open images/"//trim(adjustl(filename))//".png")
21 end subroutine graphABBTre
```

El procedimiento `searchLayer` busca una capa en el árbol por su identificador. Llama a la función `searchLayer\_Rec` para realizar la búsqueda.



```
1  function searchLayer(this, id) result(res)
2      class(abbtrees_layers), intent(in) :: this
3      type(layer), pointer :: res
4      integer, intent(in) :: id
5      type(layer) :: temp
6      temp = layer(id=id)
7      res => this%searchLayer_Rec(this%root, temp)
8  end function searchLayer
9
10 !no usar por separado
11 recursive function searchLayer_Rec(this,root, temp) result(res)
12     class(abbtrees_layers), intent(in) :: this
13     type(layer), pointer :: root
14     type(layer), intent(in) :: temp
15     class(layer), pointer :: res
16     if (.not. associated(root)) then
17         print *, "No se encontro la capa ", temp%id
18         return
19     end if
20
21     if (temp%id < root%id) then
22         res => this%searchLayer_Rec(root%left, temp)
23     else if (temp%id > root%id) then
24         res => this%searchLayer_Rec(root%right, temp)
25     else
26         res => root
27     end if
28 end function searchLayer_Rec
```

El procedimiento `leaf\_layers` imprime los identificadores de todas las capas que son hojas en el árbol.



```
1  subroutine leaf_layers(this, tmp)
2      class(abbtrees_layers), intent(in) :: this
3      type(layer), intent(in), pointer :: tmp
4      if(associated(tmp)) then
5          call this%leaf_layers(tmp%left)
6          if (.not. associated(tmp%left) .and. .not. associated(tmp%right)) then
7              write(*,"(I4)", advance='no') tmp%id
8          end if
9          call this%leaf_layers(tmp%right)
10     else
11         return
12     end if
13 end subroutine leaf_layers
```

Module_avlTree_images

Dentro del módulo, se definen dos tipos de datos: `image` y `avlTree\_images`. `image` es una estructura que contiene un identificador (`id`), una altura (`height`), un árbol binario de búsqueda (`abb`) y dos punteros a otras imágenes (`left` y `right`). `avlTree\_images` es una estructura que representa un árbol AVL de imágenes. Contiene un puntero a la imagen raíz (`root`), un contador de imágenes (`num\_images`) y una serie de procedimientos que definen las operaciones que se pueden realizar en el árbol.



```

1  type :: image
2      integer :: id
3      integer :: height
4      type(abbtrees_layers) :: abb
5      type(image), pointer :: left => null()
6      type(image), pointer :: right => null()
7  end type
8
9  type :: avlTree_images
10     type(image), pointer :: root => null()
11     integer :: num_images = 0
12     contains
13     procedure :: insertImage
14     procedure :: insertImage_rec !no usar por separado
15     procedure :: preorderAVL
16     procedure :: inorderAVL
17     procedure :: postorderAVL
18     procedure :: srl !no usar por separado
19     procedure :: srr !no usar por separado
20     procedure :: drl !no usar por separado
21     procedure :: drr !no usar por separado
22     procedure :: getheight !no usar por separado
23     procedure :: getmax !no usar por separado
24     procedure :: graphImages
25     procedure :: searchImage
26     procedure :: searchImage_rec !no usar por separado
27     procedure :: deleteImg
28     procedure :: deleteImg_rec !no usar por separado
29     procedure :: breadthFirstMatrix
30     procedure :: top_five_images_with_more_layers

```

El procedimiento `insertImage` inserta una nueva imagen en el árbol AVL. Primero, busca la imagen en el árbol. Si la imagen ya existe, se imprime un mensaje y se termina el procedimiento. Si la imagen no existe, se incrementa el contador de imágenes y se llama al procedimiento `insertImage\_rec` para insertar la imagen de forma recursiva.

```

1  subroutine insertImage(this, id,abb)
2      class(avlTree_images), intent(inout) :: this
3      integer, intent(in) :: id
4      type(abbtrees_layers), intent(in) :: abb
5      type(image), pointer :: tmp,test
6      test => this%searchImage(id)
7      if (associated(test)) then
8          print *, "La imagen ya existe"
9          return
10     end if
11     if(associated(this%root)) then
12         this%num_images = this%num_images + 1
13         call this%insertImage_rec(tmp=this%root,id=id,abb=abb)
14     else
15         this%num_images = this%num_images + 1
16         allocate(tmp)
17         tmp%height = 0
18         tmp%abb = abb
19         tmp%id = id
20         this%root => tmp
21     end if
22 end subroutine insertImage

```

El procedimiento `insertImage\_rec` inserta una imagen en el árbol de forma recursiva. Si el nodo actual no está asociado (es decir, es nulo), se crea un nuevo nodo con la imagen. Si el `id` de la imagen es igual al `id` del nodo actual, se actualiza el árbol binario de búsqueda del nodo actual y se decrementa el contador de imágenes. Si el `id` de la imagen es menor o mayor que el `id` del nodo actual, se realiza una inserción recursiva en el subárbol izquierdo o derecho, respectivamente. Después de cada inserción, se verifica si el árbol se ha desequilibrado y, en caso afirmativo, se realiza una rotación para equilibrarlo.

```

1  subroutine insertImage_rec(this,tmp, id,abb)
2      class(avlTree_images), intent(inout) :: this
3      integer, intent(in) :: id
4      type(image), pointer, intent(inout) :: tmp
5      class(abbtree_layers), intent(in) :: abb
6      integer :: r, l, m
7      if (.not. associated(tmp)) then
8          allocate(tmp)
9          tmp%height = 0
10         tmp%abb = abb
11         tmp%id = id
12     else if (id == tmp%id) then
13         tmp%abb = abb
14         this%num_images = this%num_images - 1
15     else if (id < tmp%id) then
16         call this%insertImage_rec(tmp=tmp%left,id=id,abb=abb)
17         if ((this%getheight(tmp%left) - this%getheight(tmp%right))==2) then
18             if (id < tmp%left%id) then
19                 tmp => this%srl(tmp)
20             else
21                 tmp => this%drl(tmp)
22             end if
23         end if
24     else if (id > tmp%id) then
25         call this%insertImage_rec(tmp=tmp%right,id=id,abb=abb)
26         if ((this%getheight(tmp%right) - this%getheight(tmp%left))==2) then
27             if (id > tmp%right%id) then
28                 tmp => this%srr(tmp)
29             else
30                 tmp => this%drr(tmp)
31             end if
32         end if
33     else
34
35     end if
36     r = this%getheight(tmp%right)
37     l = this%getheight(tmp%left)
38     m = this%getmax(r, l)
39     tmp%height = m + 1
40 end subroutine insertImage_rec

```

La función `getheight` devuelve la altura de un nodo. Si el nodo no está asociado (es decir, es nulo), devuelve -1. De lo contrario, devuelve la altura del nodo.



```
1  integer function getheight (this, tmp)
2      class(avlTree_images), intent(in) :: this
3      type(image), intent(in), pointer :: tmp
4      if (.not. associated(tmp)) then
5          getheight = -1
6      else
7          getheight = tmp%height
8      end if
9  end function getheight
```

Las funciones srl y srr realizan rotaciones simples a la izquierda y a la derecha, respectivamente. Estas rotaciones son operaciones fundamentales en un árbol AVL para mantenerlo equilibrado. Cada función toma como argumentos una instancia de avlTree_images y un puntero a una imagen, y devuelve un puntero a una imagen. La rotación se realiza cambiando los punteros entre las imágenes y actualizando sus alturas.



```
1  ! rotacion simple a la izquierda
2  function srl(this, t1) result(t2)
3      class(avlTree_images), intent(in) :: this
4      type(image), intent(in), pointer :: t1
5      type(image), pointer :: t2
6      t2 => t1%left
7      t1%left => t2%right
8      t2%right => t1
9      t1%height = this%getmax(this%getheight(t1%left), this%getheight(t1%right))+1
10     t2%height = this%getmax(this%getheight(t2%left), t1%height)+1
11 end function srl
12
13 ! rotacion simple a la derecha
14 function srr(this, t1) result(t2)
15     class(avlTree_images), intent(in) :: this
16     type(image), intent(in), pointer :: t1
17     type(image), pointer :: t2
18     t2 => t1%right
19     t1%right => t2%left
20     t2%left => t1
21     t1%height = this%getmax(this%getheight(t1%left), this%getheight(t1%right))+1
22     t2%height = this%getmax(this%getheight(t2%right), t1%height)+1
23 end function srr
```

Las funciones `drl` y `drr` realizan rotaciones dobles a la izquierda y a la derecha, respectivamente. Una rotación doble es una combinación de una rotación simple en una dirección seguida de una rotación simple en la dirección opuesta. Al igual que las funciones de rotación simple, estas funciones toman una instancia de `avlTree_images` y un puntero a una imagen, y devuelven un puntero a una imagen.



```
1  ! doble rotacion a la izquierda
2  function drl(this, tmp) result(res)
3      class(avlTree_images), intent(in) :: this
4      type(image), intent(in), pointer :: tmp
5      type(image), pointer :: res
6      tmp%left => this%srr(tmp%left)
7      res => this%srl(tmp)
8  end function drl
9
10 !doble rotacion a la derecha
11 function drr(this, tmp) result(res)
12     class(avlTree_images), intent(in) :: this
13     type(image), intent(in), pointer :: tmp
14     type(image), pointer :: res
15     tmp%right => this%srl(tmp%right)
16     res => this%srr(tmp)
17 end function drr
```

La función getmax devuelve el máximo de dos valores enteros. Utiliza la función merge de Fortran, que elige entre dos valores basándose en una condición.



```
1  integer function getmax(this, val1, val2)
2      class(avlTree_images), intent(in) :: this
3      integer, intent(in) :: val1, val2
4      getmax = merge(val1, val2, val1 > val2)
5  end function getmax
6
```

Las subrutinas preorderAVL, inorderAVL y postorderAVL realizan recorridos en preorden, inorden y postorden del árbol AVL, respectivamente. Cada subrutina toma una instancia de avlTree_images, un puntero a una imagen y una cola como argumentos. Si la imagen está asociada (es decir, no es nula), se añade su identificador a la cola y se realiza un recorrido recursivo en los subárboles izquierdo y derecho. El orden en que se realizan estas operaciones depende del tipo de recorrido.

```
1  subroutine preorderAVL(this, tmp,cola)
2      class(avlTree_images), intent(in) :: this
3      type(image), intent(in), pointer :: tmp
4      type(Queue), intent(inout) :: cola
5      if(associated(tmp)) then
6          call cola%enqueue(tmp%id)
7          call this%preorderAVL(tmp%left,cola)
8          call this%preorderAVL(tmp%right,cola)
9      else
10         return
11     end if
12 end subroutine preorderAVL
13
14 subroutine inorderAVL(this, tmp,cola)
15     class(avlTree_images), intent(in) :: this
16     type(image), intent(in), pointer :: tmp
17     type(Queue), intent(inout) :: cola
18     if(associated(tmp)) then
19         call this%inorderAVL(tmp%left,cola)
20         call cola%enqueue(tmp%id)
21         call this%inorderAVL(tmp%right,cola)
22     else
23         return
24     end if
25 end subroutine inorderAVL
26
27 subroutine postorderAVL(this, tmp,cola)
28     class(avlTree_images), intent(in) :: this
29     type(image), intent(in), pointer :: tmp
30     type(Queue), intent(inout) :: cola
31     if( .not. associated(tmp)) then
32         call this%postorderAVL(tmp%left,cola)
33         call this%postorderAVL(tmp%right,cola)
34         call cola%enqueue(tmp%id)
35     else
36         return
37     end if
38 end subroutine postorderAVL
```

La subrutina graphImages toma una instancia de avlTree_images y un nombre de archivo como argumentos. Esta subrutina genera un archivo .dot (un tipo de archivo utilizado para representar gráficos estructurados) para el árbol AVL de imágenes. Si el árbol está vacío, se escribe "empty" en el archivo .dot. Si el árbol no está vacío, se llama a la subrutina graphImages_rec para recorrer el árbol y escribir la información de cada nodo en el archivo .dot. Después de escribir toda la información del árbol en el archivo .dot, se cierra el archivo y se ejecuta un comando para convertir el archivo .dot en una imagen .png.

```
1  subroutine graphImages(this, filename)
2      class(avlTree_images), intent(in) :: this
3      character(len=*) , intent(in) :: filename
4      character(:),allocatable :: path
5      integer :: file
6      path = "images/"//trim(adjustl(filename))//".dot"
7      open(file, file=path, status="replace")
8      write(file, '(A)') 'digraph{'
9      if (associated(this%root)) then
10         call graphImages_rec(this%root, file)
11     else
12         write(file, '(A)') '"empty" [label="Empty Images", shape=box];'
13     end if
14     write(file, '(A)') '}'
15     close(file)
16     call execute_command_line("dot -Tpng images/"//trim(adjustl(filename))//".dot -o images/"//trim(adjustl(filename))//".png")
17     !windows
18     call system("start images\\"//trim(adjustl(filename))//".png")
19     ! linux
20     ! call system("xdg-open images\\"//trim(adjustl(filename))//".png")
21 end subroutine graphImages
```

La subrutina graphImages_rec es una subrutina recursiva que recorre el árbol AVL y escribe la información de cada nodo en el archivo .dot. Toma un puntero a una imagen y una unidad de archivo como argumentos. Si el puntero a la imagen no está asociado (es decir, es nulo), la subrutina retorna. Si el puntero a la imagen está asociado, se escribe la información de la imagen en el archivo .dot y se llama a graphImages_rec para los subárboles izquierdo y derecho.



```
1 recursive subroutine graphImages_rec(tmp, unit)
2     type(image), intent(in), pointer :: tmp
3     integer, intent(in) :: unit
4     if (.not. associated(tmp)) then
5         return
6     end if
7     write (unit, '(A,I5,A,I5,A)') ' ', tmp%id, ' [label="", tmp%id, "];'
8     if (associated(tmp%left)) then
9         write (unit, '(A,I5,A,I5,A)') ' ', tmp%id, ' -> ', tmp%left%id, ';'
10    end if
11    if (associated(tmp%right)) then
12        write (unit, '(A,I5,A,I5,A)') ' ', tmp%id, ' -> ', tmp%right%id, ';'
13    end if
14    call graphImages_rec(tmp%left, unit)
15    call graphImages_rec(tmp%right, unit)
16 end subroutine graphImages_rec
```

La función searchImage toma una instancia de avlTree_images y un id de imagen como argumentos y devuelve un puntero a la imagen con el id dado. Si la imagen no se encuentra en el árbol, la función devuelve un puntero nulo. La búsqueda se realiza llamando a la función searchImage_Rec.



```
1 function searchImage(this, id) result(res)
2     class(avlTree_images), intent(in) :: this
3     type(image), pointer :: res
4     integer, intent(in) :: id
5     res => null()
6     res => this%searchImage_Rec(this%root, id)
7 end function searchImage
```

La función searchImage_Rec es una función recursiva que busca una imagen con un id dado en el árbol AVL. Toma una instancia de avlTree_images, un puntero a una imagen y un id de imagen como argumentos y devuelve un puntero a la imagen con el id dado. Si el puntero a la imagen no está asociado o si el id de la imagen no coincide con el id dado, la función retorna un puntero nulo. Si el id de la imagen coincide con el id dado, la función retorna un puntero a la imagen.



```
1 recursive function searchImage_Rec(this,root, id) result(res)
2   class(avlTree_images), intent(in) :: this
3   type(image), pointer :: root
4   ! type(image), intent(in) :: temp
5   integer, intent(in) :: id
6   class(image), pointer :: res
7   if (.not. associated(root)) then
8     print *, "No se encontro la imagen ", id
9     res => null()
10    return
11  end if
12
13  if (id < root%id) then
14    res => this%searchImage_Rec(root%left, id)
15  else if (id > root%id) then
16    res => this%searchImage_Rec(root%right, id)
17  else
18    print *, "Se encontro la imagen ", root%id
19    res => root
20  end if
21 end function searchImage_Rec
```

La subrutina deleteImg toma una instancia de avlTree_images y un id de imagen como argumentos. Esta subrutina llama a la función deleteImg_rec para eliminar la imagen con el id dado del árbol AVL.



```
1 subroutine deleteImg(this, id)
2   class(avlTree_images), intent(inout) :: this
3   integer, intent(in) :: id
4
5   this%root => this%deleteImg_rec(this%root, id)
6 end subroutine deleteImg
```

La función `deletelmg_rec` es una función recursiva que busca la imagen con el id dado en el árbol AVL y la elimina. Toma una instancia de `avlTree_images`, un puntero a una imagen y un id de imagen como argumentos y devuelve un puntero a la raíz del árbol AVL después de la eliminación. Si el id de la imagen es menor que el id del nodo actual, la función busca en el subárbol izquierdo. Si el id de la imagen es mayor que el id del nodo actual, la función busca en el subárbol derecho. Si el id de la imagen coincide con el id del nodo actual, la función elimina el nodo. Si el nodo tiene un solo hijo o ningún hijo, simplemente se elimina el nodo. Si el nodo tiene dos hijos, se encuentra el nodo con el valor más grande en el subárbol izquierdo (o el nodo con el valor más pequeño en el subárbol derecho), se copia el valor de ese nodo en el nodo actual y luego se elimina ese nodo. Después de la eliminación, la función actualiza la altura del nodo actual y realiza rotaciones para mantener el árbol AVL equilibrado.

```

1 recursive function deleteImg_rec(this, root, id) result(res)
2   class(avlTree_images), intent(inout) :: this
3   type(image), pointer :: root
4   integer, intent(in) :: id
5   type(image), pointer :: res
6   type(image), pointer :: temp
7
8   if(.not. associated(root)) then
9     res => root
10    return
11  end if
12  if(id < root%id) then
13    root%left => this%deleteImg_rec(root%left, id)
14
15  else if(id > root%id) then
16    root%right => this%deleteImg_rec(root%right, id)
17
18  else
19    if (.not. associated(root%left)) then
20      temp => root%right
21      deallocate(root)
22      res => temp
23    else if (.not. associated(root%right)) then
24      temp => root%left
25      deallocate(root)
26      res => temp
27    else
28      call getMajorOfMinorsImages(root%left, temp)
29      root%id = temp%id
30      root%left => this%deleteImg_rec(root%left, temp%id)
31    end if
32  end if
33  res => root
34  if(.not. associated(root)) return
35  if (.not. associated(root%left) .and. .not. associated(root%right)) then
36    root%height = 0
37  else
38    if (associated(root%left) .and. associated(root%right)) then
39      root%height = this%getmax(root%left%height, root%right%height) + 1
40    else if (associated(root%left)) then
41      root%height = root%left%height + 1
42    else
43      root%height = root%right%height + 1
44    end if
45  end if
46  if(getBalance(root) > 1) then
47    if(getBalance(root%right) < 0) then
48      ! root%right => rightRotation(root%right)
49      ! root => leftRotation(root)
50      root=>this%drp(root)
51
52    else
53      ! root => leftRotation(root)
54      root => this%slr(root)
55    end if
56  end if
57  if(getBalance(root) < -1) then
58    if(getBalance(root%left) > 0) then
59      ! root%left => leftRotation(root%left)
60      ! root => rightRotation(root)
61      root=>this%drl(root)
62
63    else
64      ! root => rightRotation(root)
65      root => this%slr(root)
66    end if
67  end if
68  res => root
69 end function deleteImg_rec

```

La función `getBalance` toma un puntero a una imagen como argumento y devuelve el factor de equilibrio del nodo. El factor de equilibrio de un nodo en un árbol AVL es la altura de su subárbol derecho menos la altura de su subárbol izquierdo. Si el nodo no está asociado, el factor de equilibrio es 0. Si el nodo no tiene hijos, el factor de equilibrio es 0. Si el nodo solo tiene un hijo derecho, el factor de equilibrio es la altura negativa del hijo derecho. Si el nodo solo tiene un hijo izquierdo, el factor de equilibrio es la altura del hijo izquierdo. Si el nodo tiene dos hijos, el factor de equilibrio es la altura del hijo derecho menos la altura del hijo izquierdo.

```
1  function getBalance(root) result(res)
2      type(image),pointer, intent(in) :: root
3
4      integer :: res
5      if (.not. associated(root)) then
6          res = 0
7          return
8      end if
9      if (.not. associated(root%left) .and. .not. associated(root%right)) then
10         res = 0
11     else if (.not. associated(root%left)) then
12         res = -root%right%height
13     else if (.not. associated(root%right)) then
14         res = root%left%height
15     else
16         res = root%right%height - root%left%height
17     end if
18
19 end function getBalance
20
```

La subrutina `getMajorOfMinorsImages` es una función recursiva que busca el nodo más grande en el subárbol derecho de un nodo dado. Toma un puntero a una imagen como argumento y devuelve un puntero a la imagen con el valor más grande. Si el nodo actual tiene un hijo derecho, la función llama a sí misma con el hijo derecho como argumento. Si el nodo actual no tiene un hijo derecho, entonces es el nodo con el valor más grande en el subárbol y la función devuelve un puntero a este nodo.



```
1 recursive subroutine getMajorOfMinorsImages(root, major)
2   type(image), pointer :: root, major
3   if (associated(root%right)) then
4     call getMajorOfMinorsImages(root%right, major)
5   else
6     major => root
7   end if
8 end subroutine getMajorOfMinorsImages
```

La subrutina `breadthFirstMatrix` realiza un recorrido en amplitud de un árbol AVL de imágenes y almacena los detalles de cada pixel en una matriz. Toma una instancia de `avlTree\_images`, una matriz y un id de imagen como argumentos. La función busca la imagen con el id dado en el árbol AVL y si no la encuentra, imprime un mensaje de error y termina. Si encuentra la imagen, realiza un recorrido en amplitud del árbol AVL de capas de la imagen. Para cada capa, recorre la lista de pixels de la capa y almacena los detalles de cada pixel en la matriz.

```

1  subroutine breadthFirstMatrix(this,actualMatrix,idImage)
2      class(avlTree_images), intent(in) :: this
3      type(matrix), intent(inout) :: actualMatrix
4      integer, intent(in) :: idImage
5      type(image), pointer :: img_temp
6      type(layer), pointer :: actualLayer
7      type(abbtree_layers) :: layersTree
8      type(pixel), pointer :: actualPixel
9      type(Queue) :: queue
10     character(7) :: color
11     integer :: i,j,id
12     img_temp => this%searchImage(idImage)
13     ! layersTree = img_temp%abb
14     if (.not. associated(img_temp)) then
15         print *, "No se encontro la imagen"
16         return
17     end if
18     actualLayer => img_temp%abb%root
19     id = actualLayer%id
20     call queue%enqueue(id)
21     do while (.not. queue%isEmpty())
22         call queue%printQueue()
23         id = queue%dequeue()
24         actualLayer => img_temp%abb%searchLayer(id)
25         ! este es el visit()
26         actualPixel => actualLayer%pixels%head
27         do while (associated(actualPixel))
28             i=actualPixel%row
29             j=actualPixel%col
30             color = trim(adjustl(actualPixel%color))
31             print *, "fila: ", i, " columna: ", j, " color: ", color
32             call actualMatrix%insert(i=i,j=j,color=color)
33             actualPixel => actualPixel%next
34         end do
35         !
36         if (associated(actualLayer%left)) then
37             id = actualLayer%left%id
38             call queue%enqueue(id)
39         end if
40         if (associated(actualLayer%right)) then
41             id = actualLayer%right%id
42             call queue%enqueue(id)
43         end if
44     end do
45 end subroutine breadthFirstMatrix

```

La subrutina `top\_five\_images\_with\_more\_layers` imprime las cinco imágenes con más capas en un árbol AVL de imágenes. Toma una instancia de `avlTree\_images`, una cola y un número de imágenes como argumentos. La función desencola los ids de las imágenes de la cola y busca cada imagen en el árbol AVL. Almacena el id de cada imagen y el número de capas de cada imagen en dos arrays. Luego ordena los dos arrays en orden ascendente por el número de capas. Finalmente, imprime los ids de las imágenes y el número de capas de las cinco imágenes con más capas.

```
1  subroutine top_five_images_with_more_layers(this,cola,num_images)
2      class(avlTree_images), intent(in) :: this
3      type(queue), intent(inout) :: cola
4      integer, intent(in) :: num_images
5      type(image), pointer :: img_temp
6      type(queue) :: cola_aux
7      character(:), allocatable :: text
8      integer :: array(1:num_images),array_sizes(1:num_images)
9      integer :: i,j,key,key2,tam
10     logical :: fin
11     fin = .false.
12     call cola%printQueue()
13     if (.not. associated(this%root)) then
14         print *, "No hay imagenes"
15         return
16     end if
17     do i = 1, num_images
18         if (i > num_images) then
19             fin = .true.
20             exit
21         end if
22         array(i) = cola%dequeue()
23         img_temp => this%searchImage(array(i))
24         if (.not. associated(img_temp)) then
25             print *, "No se encontro la imagen"
26             return
27         end if
28         array_sizes(i) = img_temp%abb%num_layers
29     end do
30     do i = 2, num_images
31         key = array_sizes(i)
32         key2 = array(i)
33         j = i - 1
34         do while (j>=1 .and. array_sizes(j) > key)
35             array_sizes(j+1) = array_sizes(j)
36             array(j+1) = array(j)
37             j = j - 1
38         end do
39         array_sizes(j+1) = key
40         array(j+1) = key2
41     end do
42     do i = 1, num_images
43         print *, "Imagen: ", array(num_images-i+1), " Capas: ", array_sizes(num_images-i+1)
44     end do
45 end subroutine top_five_images_with_more_layers
```

Module_Queue

Este código define un módulo en Fortran llamado `module\_Queue` que implementa una estructura de datos de cola. La cola está compuesta por nodos, cada uno de los cuales tiene un `id` de tipo entero, un `id\_8` de tipo entero de 8 bytes y un puntero `next` al siguiente nodo en la cola.



```
1  type :: node
2      integer :: id
3      integer(kind=8) :: id_8
4      type(node), pointer :: next => null()
5  end type node
```

La cola tiene dos punteros, `head` y `tail`, que apuntan al primer y último nodo de la cola, respectivamente. La cola también tiene varias subrutinas y funciones que permiten manipular la cola.



```
1  type :: queue
2      type(node), pointer :: head => null()
3      type(node), pointer :: tail => null()
4      contains
5      procedure :: enqueue
6      procedure :: enqueue_8
7      procedure :: dequeue
8      procedure :: dequeue_8
9      procedure :: isEmpty
10     procedure :: printQueue
11     procedure :: printQueue_8
12     procedure :: cleanQueue
13     procedure :: getQueueSize
14 end type queue
```

La subrutina `enqueue` añade un nuevo nodo al final de la cola. Toma una instancia de la cola y un `id` de tipo entero como argumentos. Crea un nuevo nodo, asigna el `id` al nodo y añade el nodo al final de la cola. Si la cola está vacía, el nuevo nodo se convierte en el primer y último nodo de la cola.



```
1  subroutine enqueue(this, id)
2      class(queue), intent(inout) :: this
3      integer, intent(in) :: id
4      type(node), pointer :: tmp
5      allocate(tmp)
6      tmp%id = id
7      if (.not. associated(this%head)) then
8          this%head => tmp
9          this%tail => tmp
10     else
11         this%tail%next => tmp
12         this%tail => tmp
13     end if
14 end subroutine enqueue
```

La subrutina `enqueue\_8` hace lo mismo que `enqueue`, pero toma un `id` de tipo entero de 8 bytes como argumento.



```
1  subroutine enqueue_8(this, id)
2      class(queue), intent(inout) :: this
3      integer(kind=8), intent(in) :: id
4      type(node), pointer :: tmp
5      allocate(tmp)
6      tmp%id_8 = id
7      if (.not. associated(this%head)) then
8          this%head => tmp
9          this%tail => tmp
10     else
11         this%tail%next => tmp
12         this%tail => tmp
13     end if
14 end subroutine enqueue_8
```

La función `dequeue` elimina el primer nodo de la cola y devuelve su `id`. Si la cola está vacía, no hace nada.



```
1  function dequeue(this) result(id)
2      class(queue), intent(inout) :: this
3      type(node), pointer :: tmp
4      integer :: id
5      if (associated(this%head)) then
6          id = this%head%id
7          tmp => this%head
8          this%head => this%head%next
9          deallocate(tmp)
10     end if
11 end function dequeue
```

La función `dequeue\_8` hace lo mismo que `dequeue`, pero devuelve un `id` de tipo entero de 8 bytes.



```
1  function dequeue_8(this) result(id)
2      class(queue), intent(inout) :: this
3      type(node), pointer :: tmp
4      integer(kind=8) :: id
5      if (associated(this%head)) then
6          id = this%head%id_8
7          tmp => this%head
8          this%head => this%head%next
9          deallocate(tmp)
10     end if
11 end function dequeue_8
```

La función `isEmpty` comprueba si la cola está vacía y devuelve un valor lógico.



```
1  function isEmpty(this) result(res)
2      class(queue), intent(in) :: this
3      logical :: res
4      res = .not. associated(this%head)
5  end function isEmpty
```

Las subrutinas `printQueue` y `printQueue\_8` imprimen todos los `id` y `id\_8` de los nodos de la cola, respectivamente.



```
1  subroutine printQueue(this)
2      class(queue), intent(in) :: this
3      type(node), pointer :: tmp
4      tmp => this%head
5      do while (associated(tmp))
6          write (*, '(A,I3)', advance='no') " ",tmp%id
7          tmp => tmp%next
8      end do
9      print *, ""
10 end subroutine printQueue
11
12 subroutine printQueue_8(this)
13     class(queue), intent(in) :: this
14     type(node), pointer :: tmp
15     tmp => this%head
16     do while (associated(tmp))
17         write (*, '(A,I3)', advance='no') " ",tmp%id_8
18         tmp => tmp%next
19     end do
20     print *, ""
21 end subroutine printQueue_8
```

La subrutina `cleanQueue` elimina todos los nodos de la cola.



```
1  subroutine cleanQueue(this)
2      class(queue), intent(inout) :: this
3      type(node), pointer :: tmp
4      do while (associated(this%head))
5          tmp => this%head
6          this%head => this%head%next
7          deallocate(tmp)
8      end do
9  end subroutine cleanQueue
```

La función `getQueueSize` devuelve el número de nodos en la cola.



```
1  function getQueueSize(this) result(size)
2      class(queue), intent(in) :: this
3      type(node), pointer :: tmp
4      integer :: size
5      size = 0
6      tmp => this%head
7      do while (associated(tmp))
8          size = size + 1
9          tmp => tmp%next
10     end do
11 end function getQueueSize
```

Module_Matrix

Este modulo define tres tipos de datos en Fortran: `node\_val`, `node` y `matrix`.

El tipo `node\_val` tiene dos campos lógicos: `exists`, que se inicializa en `.false.` , y `value`.



```
1  type :: node_val
2      private
3      logical :: exists = .false.
4      logical :: value
5  end type node_val
```

El tipo `node` representa un nodo en una matriz dispersa y tiene cinco campos. Los campos `i` y `j` son enteros que representan las coordenadas del nodo en la matriz. El campo `color` es una cadena de caracteres de longitud 7. Los campos `up`, `down`, `right` y `left` son punteros a otros nodos en la matriz. Estos punteros representan las conexiones del nodo con sus vecinos en las direcciones arriba, abajo, derecha e izquierda, respectivamente.



```
1  type :: node
2      private
3      integer :: i, j
4      character(7) :: color
5      type(node), pointer :: up => null()
6      type(node), pointer :: down => null()
7      type(node), pointer :: right => null()
8      type(node), pointer :: left => null()
9  end type node
```

El tipo `matrix` representa una matriz dispersa y tiene tres campos. El campo `root` es un puntero al nodo raíz de la matriz. Los campos `width` y `height` son enteros que representan el ancho y la altura de la matriz, respectivamente. El tipo `matrix` también contiene varias subrutinas y funciones que permiten manipular la matriz.



```
1  type, public :: matrix
2      type(node), pointer :: root
3      integer :: width = 0
4      integer :: height = 0
5  contains
6      procedure :: insert
7      procedure :: insertRowHeader
8      procedure :: insertColumnHeader
9      procedure :: insertInRow
10     procedure :: insertInColumn
11     procedure :: searchRow
12     procedure :: searchColumn
13     procedure :: nodeExists
14     procedure :: print
15     procedure :: printColumnHeaders
16     procedure :: getValue
17     procedure :: graphMatrix
18     procedure :: graphTable
19     procedure :: cleanMatrix
20 end type
```

La subrutina `insert` inserta un nuevo nodo en la matriz. Toma una instancia de la matriz, las coordenadas `i` y `j` del nuevo nodo y su `color` como argumentos. Crea un nuevo nodo y lo asigna a las coordenadas `i` y `j`. Si la matriz está vacía, crea un nodo raíz. Luego busca la fila y la columna donde se insertará el nuevo nodo. Si la fila o la columna no existen, las crea. Finalmente, inserta el nuevo nodo en la fila y la columna correspondientes.



```
1  subroutine insert(this, i, j, color)
2      class(matrix), intent(inout) :: this
3      integer, intent(in) :: i
4      integer, intent(in) :: j
5      character(7), intent(in) :: color
6
7      type(node), pointer :: new
8      type(node), pointer :: row
9      type(node), pointer :: column
10     allocate(new)
11     new = node(i=i, j=j, color=color)
12     if(.not. associated(this%root)) then
13         allocate(this%root)
14         print *, "Creando nodo raiz"
15         this%root = node(i=-1, j=-1, color="#FFFFFF")
16     end if
17     row => this%searchRow(i)
18     column => this%searchColumn(j)
19     if(j > this%width) this%width = j
20     if(i > this%height) this%height = i
21     if(.not. this%nodeExists(new)) then
22         if(.not. associated(column)) then
23             column => this%insertColumnHeader(j)
24         end if
25         if(.not. associated(row)) then
26             row => this%insertRowHeader(i)
27         end if
28         call this%insertInColumn(new, row)
29         call this%insertInRow(new, column)
30     end if
31 end subroutine insert
```

La función `searchColumn` busca una columna en la matriz. Toma una instancia de la matriz y la coordenada `j` de la columna como argumentos. Comienza en el nodo raíz de la matriz y sigue el puntero `right` hasta que encuentra la columna o llega al final de la matriz.



```
1  function searchColumn(this, j) result(actual)
2      class(matrix), intent(in) :: this
3      integer, intent(in) :: j
4
5      type(node), pointer :: actual
6      actual => this%root
7      do while(associated(actual))
8          if(actual%j == j) then
9              return
10         end if
11         actual => actual%right
12     end do
13     actual => null()
14 end function searchColumn
```

La función `searchRow` hace lo mismo que `searchColumn`, pero busca una fila en la matriz. Toma una instancia de la matriz y la coordenada `i` de la fila como argumentos. Comienza en el nodo raíz de la matriz y sigue el puntero `down` hasta que encuentra la fila o llega al final de la matriz.



```
1  function searchRow(this, i) result(actual)
2      class(matrix), intent(inout) :: this
3      integer, intent(in) :: i
4
5      type(node), pointer :: actual
6      actual => this%root
7      do while(associated(actual))
8          if(actual%i == i) then
9              return
10         end if
11         actual => actual%down
12     end do
13     actual => null()
14 end function searchRow
```

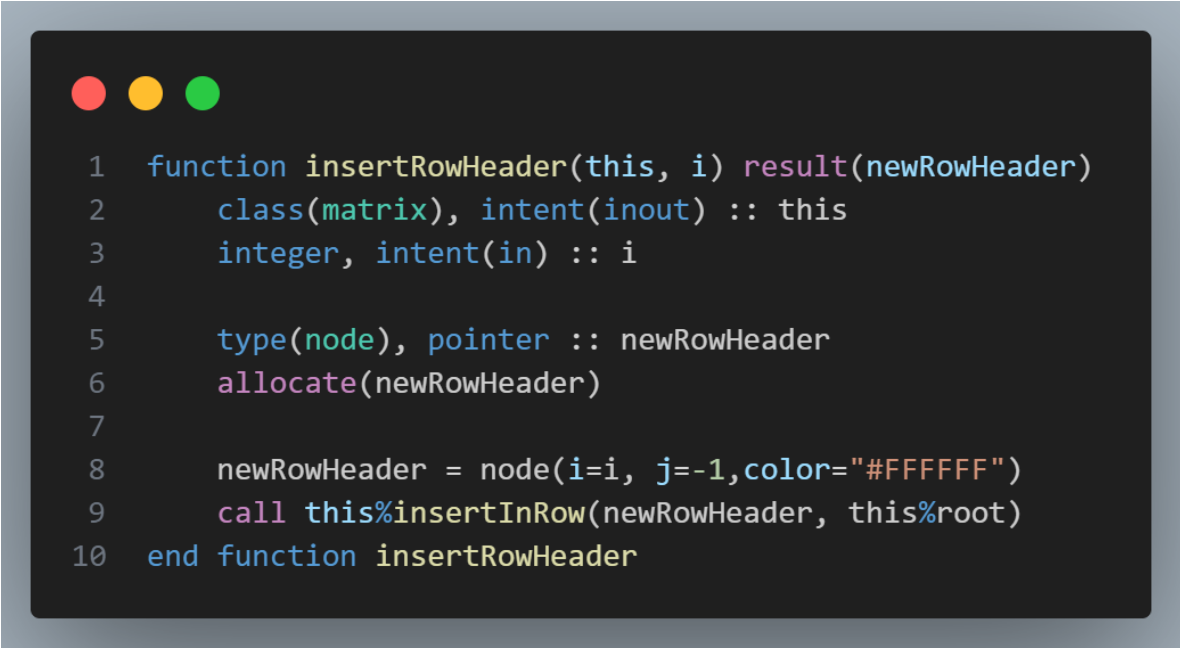
La función `nodeExists` comprueba si un nodo existe en la matriz. Toma una instancia de la matriz y un nodo como argumentos. Comienza en el nodo raíz de la matriz y sigue los punteros `down` y `right` hasta que encuentra el nodo o llega al final de la matriz. Si encuentra el nodo, actualiza su `color` y devuelve `.true.`. Si no encuentra el nodo, devuelve `.false.`.



```
1  function nodeExists(this, new) result(exists)
2      class(matrix), intent(inout) :: this
3      type(node), pointer :: new
4
5      logical :: exists
6      type(node), pointer :: rowHeader
7      type(node), pointer :: column
8      rowHeader => this%root
9      exists = .false.
10
11     do while(associated(rowHeader))
12         if(rowHeader%i == new%i) then
13             column => rowHeader
14             do while(associated(column))
15                 if(column%j == new%j) then
16                     column%color = new%color
17                     exists = .true.
18                     return
19                 end if
20                 column => column%right
21             end do
22             return
23         end if
24         rowHeader => rowHeader%down
25     end do
26     return
27 end function nodeExists
```

La función `insertRowHeader` crea un nuevo encabezado de fila en la matriz. Toma una instancia de la matriz y un índice de fila `i` como argumentos. Crea un nuevo nodo con el índice de fila `i`, un índice de columna de `-1` (indicando que es un encabezado de fila) y un

color blanco. Luego llama a la subrutina `insertInRow` para insertar el nuevo encabezado de fila en la matriz.



```
1 function insertRowHeader(this, i) result(newRowHeader)
2   class(matrix), intent(inout) :: this
3   integer, intent(in) :: i
4
5   type(node), pointer :: newRowHeader
6   allocate(newRowHeader)
7
8   newRowHeader = node(i=i, j=-1, color="#FFFFFF")
9   call this%insertInRow(newRowHeader, this%root)
10 end function insertRowHeader
```

La subrutina `insertInRow` inserta un nuevo nodo en una fila específica de la matriz. Toma un nuevo nodo y un encabezado de fila como argumentos. Comienza en el encabezado de fila y recorre la fila hasta que encuentra el lugar correcto para insertar el nuevo nodo, basándose en el índice de fila del nuevo nodo. Si el lugar correcto para insertar el nuevo nodo es al final de la fila, lo inserta allí.



```

1  subroutine insertInRow(this, new, rowHeader)
2      class(matrix), intent(inout) :: this
3      type(node), pointer :: new
4      type(node), pointer :: rowHeader
5
6      type(node), pointer :: actual
7      actual => rowHeader
8
9      do while(associated(actual%down))
10         if(new%i < actual%down%i .and. new%i > actual%i) then
11             new%down => actual%down
12             new%up => actual
13             actual%down%up => new
14             actual%down => new
15             exit
16         end if
17         actual => actual%down
18     end do
19
20     if(.not. associated(actual%down)) then
21         actual%down => new
22         new%up => actual
23     end if
24 end subroutine insertInRow

```

La función `insertColumnHeader` hace lo mismo que `insertRowHeader`, pero para encabezados de columna. Crea un nuevo encabezado de columna con un índice de fila de `1` (indicando que es un encabezado de columna), el índice de columna `j` y un color blanco. Luego llama a la subrutina `insertInColumn` para insertar el nuevo encabezado de columna en la matriz.



```
1  function insertColumnHeader(this, j) result(newColumnHeader)
2      class(matrix), intent(inout) :: this
3      integer, intent(in) :: j
4
5      type(node), pointer :: newColumnHeader
6      allocate(newColumnHeader)
7
8      newColumnHeader = node(i=-1, j=j, color="#FFFFFF")
9      call this%insertInColumn(newColumnHeader, this%root)
10 end function insertColumnHeader
```

La subrutina `insertInColumn` hace lo mismo que `insertInRow`, pero para columnas. Inserta un nuevo nodo en una columna específica de la matriz, basándose en el índice de columna del nuevo nodo. Si el lugar correcto para insertar el nuevo nodo es al final de la columna, lo inserta allí.



```
1  subroutine insertInColumn(this, new, columnHeader)
2      class(matrix), intent(inout) :: this
3      type(node), pointer :: new
4      type(node), pointer :: columnHeader
5
6      type(node), pointer :: actual
7      actual => columnHeader
8      do while(associated(actual%right))
9          if(new%j < actual%right%j .and. new%j > actual%j) then
10              new%right => actual%right
11              new%left => actual
12              actual%right%left => new
13              actual%right => new
14              exit
15          end if
16          actual => actual%right
17      end do
18
19      if(.not. associated(actual%right)) then
20          actual%right => new
21          new%left => actual
22      end if
23  end subroutine insertInColumn
```

`print`: Esta subrutina imprime la matriz en la consola. Recorre cada elemento de la matriz y, si el elemento está asociado (es decir, no es nulo), imprime 1; de lo contrario, imprime un espacio en blanco.



```

1  subroutine print(this)
2      class(matrix), intent(inout) :: this
3      integer :: i
4      integer :: j
5      type(node), pointer :: aux
6      type(node), pointer :: val
7      aux => this%root%down
8
9      call this%printColumnHeaders()
10
11     do i = 0, this%height
12         print *, ""
13         write(*, fmt='(I3)', advance='no') i
14         do j = 0, this%width
15             val => this%getValue(i,j)
16             if(associated(val)) then
17                 write(*, fmt='(L3)', advance='no') 1
18             else
19                 write(*, fmt='(A3)', advance='no') " "
20             end if
21         end do
22     end do
23 end subroutine print

```

`printColumnHeaders`: Esta subrutina imprime los encabezados de las columnas de la matriz en la consola.



```
1  subroutine printColumnHeaders(this)
2      class(matrix), intent(in) :: this
3      integer :: j
4
5      do j=-1, this%width
6          write(*, fmt='(I3)', advance='no') j
7      end do
8  end subroutine printColumnHeaders
```

`getValue`: Esta función devuelve el valor de un elemento específico de la matriz en las posiciones `i` y `j`. Si el elemento no existe, imprime un mensaje y devuelve un puntero nulo.

```

1  function getValue(this, i, j) result(val)
2      class(matrix), intent(in) :: this
3      integer, intent(in) :: i
4      integer, intent(in) :: j
5
6      type(node), pointer :: rowHeader
7      type(node), pointer :: column
8      type(node), pointer :: val
9      rowHeader => this%root
10     val => null()
11     do while(associated(rowHeader))
12         if(rowHeader%i == i) then
13             column => rowHeader
14             do while(associated(column))
15                 if(column%j == j) then
16                     print *, "Nodo encontrado"
17                     val => column
18                     return
19                 end if
20                 column => column%right
21             end do
22             print *, "No existe el nodo, (getValue Matrix)"
23             return
24         end if
25         rowHeader => rowHeader%down
26     end do
27 end function getValue

```

`graphMatrix`: Esta subrutina genera un gráfico de la matriz y lo guarda como una imagen. Utiliza la herramienta Graphviz para generar el gráfico y luego lo guarda como un archivo PNG.

```

1  subroutine graphMatrix(this,filename)
2      class(matrix), intent(in) :: this
3
4      integer :: unit,i
5      character(len=10) :: str_i,str_j,str_i_aux,str_j_aux
6      character(len=150) :: node_dec
7      character(len=20) :: nombre
8      character(len=*) ,intent(in) :: filename
9
10     character(:, allocatable) :: rank
11     character(:, allocatable) :: conexion
12     character(:, allocatable) :: conexionRev
13     type(node), pointer :: fila_aux
14     type(node), pointer :: columna_aux
15     fila_aux => this%root
16
17     open(unit, file="images/"//trim(adjustl(filename))//".dot",status="replace")
18
19     write(unit, *) "digraph Matrix {"
20     write(unit, *) 'node[shape = "box"]'
21
22     do while (associated(fila_aux))
23         rank = "rank=same"
24         columna_aux => fila_aux
25         do while(associated(columna_aux))
26             write(str_i, '(I10)') columna_aux%i + 1
27             write(str_j, '(I10)') columna_aux%j + 1
28             nombre = "Nodo"//trim(adjustl(str_i))//"_ " //trim(adjustl(str_j))//""
29
30             if (columna_aux%i == -1 .and. columna_aux%j == -1) then
31                 node_dec = trim(adjustl(nombre))//[label = "root", group="//trim(adjustl(str_i))//"]'
32
33             else if(columna_aux%i == -1) then
34                 write(str_i_aux, '(I10)') columna_aux%j
35                 node_dec = trim(adjustl(nombre))//[label = ""//trim(adjustl(str_i_aux))
36                 node_dec = trim(adjustl(node_dec))//", group="//trim(adjustl(str_i))//"]'
37
38             else if(columna_aux%j == -1) then
39                 write(str_i_aux, '(I10)') columna_aux%i
40                 node_dec = trim(adjustl(nombre))//[label = ""//trim(adjustl(str_i_aux))
41                 node_dec = trim(adjustl(node_dec))//", group="//trim(adjustl(str_i))//"]'
42
43             else
44                 node_dec = trim(adjustl(nombre))//[label = ""// "
45                 node_dec = trim(adjustl(node_dec))//", style = filled, fillcolor= ""//&
46                 & trim(adjustl(columna_aux%color))//"" group=""// &
47                 & trim(adjustl(str_i))//"]'
48
49             end if
50             write(unit, *) node_dec
51
52             if(associated(columna_aux%right)) then
53                 conexion = "Nodo"//trim(adjustl(str_i))//"_ " //trim(adjustl(str_j))//""->'
54
55                 write(str_i_aux, '(I10)') columna_aux%right%i + 1
56                 write(str_j_aux, '(I10)') columna_aux%right%j + 1
57
58                 conexion = conexion//"Nodo"//trim(adjustl(str_i_aux))//"_ " //trim(adjustl(str_j_aux))//""
59                 conexionRev = conexion//[dir = back]'
60                 write(unit, *) conexion
61                 write(unit, *) conexionRev
62             end if
63
64             if(associated(columna_aux%down)) then
65                 conexion = "Nodo"//trim(adjustl(str_i))//"_ " //trim(adjustl(str_j))//""->'
66
67                 write(str_i_aux, '(I10)') columna_aux%down%i + 1
68                 write(str_j_aux, '(I10)') columna_aux%down%j + 1
69
70                 conexion = conexion//"Nodo"//trim(adjustl(str_i_aux))//"_ " //trim(adjustl(str_j_aux))//""
71                 conexionRev = conexion//[dir = back]'
72                 write(unit, *) conexion
73                 write(unit, *) conexionRev
74             end if
75
76             rank = rank//";"//trim(adjustl(str_i))//"_ " //trim(adjustl(str_j))//""
77             columna_aux => columna_aux%right
78         end do
79         rank = rank//'}'
80         write(unit, *) rank
81
82         fila_aux => fila_aux%down
83     end do
84     write(unit, *) "}"
85     close(unit)
86     !el -Gnslimit=2 indica el numero de iteraciones que hará graphviz para ajustar los nodos
87     call execute_command_line("dot -Gnslimit=2 -Tpng images/"//trim(adjustl(filename))//&
88     &"dot -o images/"//trim(adjustl(filename))//".png", exitstat=i)
89
90     if (i == 1) then
91         print *, "Ocurrió un error al momento de crear la imagen"
92     else
93         print *, "La imagen fue generada exitosamente"
94         !windows
95         call system("start images\\"//trim(adjustl(filename))//".png")
96         ! linux
97         ! call system("xdg-open images/"//trim(adjustl(filename))//".png")
98     end if
99 end subroutine graphMatrix

```

`graphTable`: Similar a `graphMatrix`, esta subrutina genera un gráfico de la matriz en forma de tabla y lo guarda como una imagen.

```
1  subroutine graphTable(this, filename,text)
2      class(matrix), intent(in) :: this
3      character(len=*), intent(in) :: filename,text
4      character(:),allocatable :: path
5      integer :: i, j,unit
6      type(node), pointer :: val
7      path = "images/"//trim(adjustl(filename))//".dot"
8      ! Open the DOT file for writing
9      open(unit, file=path, status='replace')
10     ! Write the DOT code to the file
11     write(unit, '(A)') "digraph{"
12     write(unit, '(A)') "  node [ shape=plaintext fontname=Helvetica ]"
13     write(unit, '(A)') ""
14     write(unit, '(A)') "  a [ label="" // trim(adjustl(text)) // "" ]"
15     write(unit, '(A)') "  b [ label = <"
16     write(unit, '(A)') "    <table border=""0"" cellborder=""0"" cellspacing=""0"" bgcolor=""white"">"
17     do i = 0, this%height
18         write(unit, '(A)') "    <tr>"
19         do j = 0, this%width
20             val => this%getValue(i, j)
21             if (.not. associated(val)) then
22                 write(unit, '(A)') "        <td></td>"
23             else
24                 write(unit, '(A)') "        <td bgcolor="" // trim(val%color) // ""></td>"
25             end if
26         end do
27         write(unit, '(A)') "    </tr>"
28     end do
29     write(unit, '(A)') "  </table>"
30     write(unit, '(A)') "  > ]"
31     write(unit, '(A)') ""
32     write(unit, '(A)') "}"
33     close(unit)
34     call system("dot -Gnslimit=2 -Tpng -o ""//images/"// trim(filename)//".png"&
35     & // " " // "images/"//trim(adjustl(filename))//".dot")
36     call system("start " // "images/"// trim(filename) // ".png")
37     !windows
38     ! call system("start images\\"//trim(adjustl(filename))//".png")
39     ! linux
40     ! call system("xdg-open images\\"//trim(adjustl(filename))//".png")
41 end subroutine graphTable
```

`cleanMatrix`: Esta subrutina limpia la matriz, liberando la memoria de todos los elementos y estableciendo la altura y la anchura de la matriz a 0.

```
1  subroutine cleanMatrix(this)
2      class(matrix), intent(inout) :: this
3      type(node), pointer :: rowHeader
4      type(node), pointer :: column
5      type(node), pointer :: aux
6      type(node), pointer :: aux2
7      rowHeader => this%root
8      column => rowHeader%right
9      do while(associated(column))
10         print *, "limpiando columna: ", column%i, "-", column%j
11         aux => column
12         column => column%right
13         deallocate(aux)
14     end do
15     rowHeader => rowHeader%down
16     do while(associated(rowHeader))
17         column => rowHeader%right
18         do while(associated(column))
19             print *, "limpiando columna: ", column%i, "-", column%j
20             aux => column
21             column => column%right
22             deallocate(aux)
23         end do
24         aux2 => rowHeader
25         rowHeader => rowHeader%down
26         print *, "limpiando fila: ", aux2%i, "-", aux2%j
27         deallocate(aux2)
28     end do
29     deallocate(this%root)
30     this%width = 0
31     this%height = 0
32     this%root => null()
33 end subroutine cleanMatrix
```